Incorporating high-accuracy, finite-volume shock stabilization methods into a quantum algorithm for nonlinear partial differential equations

Ryan Vogt,[1] Harold Berjamin,[2] Hunter Rouillard,[3] Edward Raff,[4,*] Priyanka Ranade,[1] Torstein Collett,[5] Roberto D'Angelo-Cosme,[1] James Holt,[1] and Frank Gaitan[3,†]

[1]*Laboratory for Physical Sciences, 5520 Research Park Drive, Baltimore, Maryland 21228, USA*
[2]*School of Mathematical and Statistical Sciences, University of Galway, University Road, Galway, Ireland*
[3]*Laboratory for Physical Sciences, 8050 Greenmead Drive, College Park, Maryland 20740, USA*
[4]*Booz Allen Hamilton, 308 Sentinel Drive, Annapolis Junction, Maryland 20701, USA*
[5]*ManTech International, 2251 Corporate Park Drive, Herndon, Virginia 20171, USA*

(Received 18 September 2024; revised 26 April 2025; accepted 28 October 2025; published 17 November 2025)

Conservation laws are of fundamental importance in science and engineering, and so are ubiquitous in applications. For continuum systems such as classical fluids and plasmas, they give rise to coupled systems of nonlinear partial differential equations (PDEs) whose solutions can develop discontinuities such as shocks. Finite-volume (FV) methods are an important tool for finding approximate solutions to such PDEs as they are based on an integral formulation of the conservation laws that can handle the presence of discontinuities. Weighted essentially nonoscillatory (WENO) methods are a class of high-resolution methods that can (i) produce solutions of arbitrarily high-order accuracy in regions where the solution is smooth, while (ii) simultaneously quashing the development of nonphysical oscillations in the vicinity of discontinuities that often cause numerical simulations to fail. In this paper we show how FV and WENO methods can be incorporated into a quantum algorithm for solving systems of nonlinear PDEs (QPDE) introduced by one of the authors. We numerically simulate application of the modified QPDE algorithm to three verification problems. The first has a smooth solution, while the other two are well-known one-dimensional Riemann problems whose exact solutions contain moving shocks, contact discontinuities, and rarefaction waves. For all three problems excellent agreement is found between the results of the QPDE simulations and the exact solutions.

DOI: 10.1103/9phc-5m7b

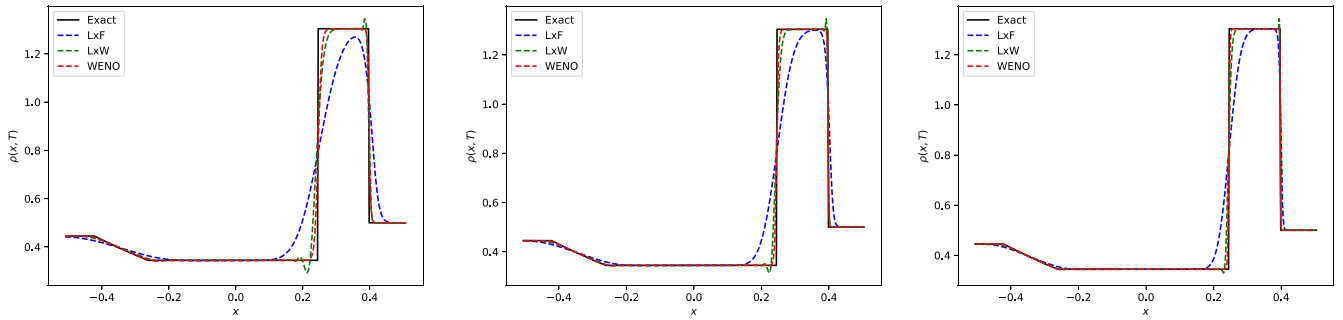
I. INTRODUCTION

Because of their fundamental importance, conservation laws appear widely in science and engineering applications. For a physical system such as a classical fluid or a plasma that is represented by a continuum system, the conservation laws give rise to coupled systems of nonlinear partial differential equations (PDEs). For example, the Navier-Stokes equations are a nonlinear system of PDEs governing the dynamics of a fluid that express the conservation of mass, linear momentum, and energy. The nonlinearities give rise to important physical effects such as shock waves, which in the limit of negligible viscosity, are discontinuities in the fluid flow.

Recently, a quantum algorithm for solving systems of nonlinear PDEs that provides a quadratic quantum speedup was introduced [1,2]. Using numerical simulation, the quantum PDE (QPDE) algorithm has been successfully applied to problems in (i) computational fluid dynamics [1,3], (ii) tephra dispersal from a submarine volcanic eruption [4], and (iii) nonlinear radiation diffusion [5,6]. In these applications finite-difference (FD) methods were used to discretize space, thereby reducing the spatial continuum to a finite number of degrees of freedom. This is essential as a computer, classical or quantum, is necessarily limited to finite computational resources. Although FD methods continue to be popular, finite-volume (FV) methods [7] have important advantages for systems of nonlinear PDEs derived from conservation laws. This is because they rely on an integral formulation of the conservation laws that remains mathematically sound even in the presence of discontinuities. Consequently, it is of interest to incorporate FV methods into the QPDE algorithm.

Euler's equations embody the conservation laws for a compressible gas with negligible viscosity and heat flow, and have long provided fertile ground for the testing of FV methods. They are a system of nonlinear PDEs capable of developing discontinuous solutions [7]. Godunov [8] introduced a method for solving Euler's equations that represents conserved quantities by spatially piecewise constant functions that specify the conserved quantities' average over a FV computational cell. Evolution of the conserved quantities over a time step is implemented by exactly solving a local Riemann problem at each cell interface and then calculating new cell averages. This process is repeated as often as necessary. Godunov's

*Present address: CrowdStrike, 206 E. 9th St, Suite 1400, Austin, Texas 78701, USA.
†Contact author: fgaitan@lps.umd.edu



(a) $N_x = 256$ (b) $N_x = 512$ (c) $N_x = 1024$

FIG. 1. Comparing the results of various FV methods for the mass density ρ against the exact solution of Verification Problem 3. For the verification problem considered in Sec. V, we plot the approximate solutions for the mass density ρ produced by the Lax-Friedrichs (LxF), Lax-Wendroff (LxW), and WENO methods against the exact solution for three spatial discretizations. (a)–(c) plot the results for spatial discretizations containing 256, 512, and 1024 computational cells, respectively. The exact solution corresponds to a shock moving to the right, followed by a contact discontinuity also moving to the right, and a rarefaction wave moving to the left. The plots are a snapshot of this motion at time $T = 0.16$.

method is well suited for the numerical solution of nonlinear PDEs which can generate discontinuities. Similar to the Lax-Friedrichs method [9], it is first-order accurate and highly dispersive, leading to a smoothing out of large gradients absent a fine spatial discretization. On the other hand, second-order methods such as the Lax-Wendroff method [10] often produce nonphysical oscillations near discontinuities. To mitigate such numerical artifacts it is common to combine first-order and second-order methods based on slope or flux limiters [7,11], although this reduces accuracy for smooth solutions below second order.

Various strategies have been proposed to increase the order of accuracy of FV methods, while at the same time suppressing numerical artifacts. Our focus will be on weighted effectively nonoscillatory (WENO) methods [12–14] that are based on a combination of spatial polynomial approximations of high-order accuracy that simultaneously quash nonphysical oscillations near discontinuities. Time stepping in WENO methods is often implemented using a third- or fourth-order Runge-Kutta method.

To illustrate these remarks, Fig. 1 shows the mass density ρ for the verification problem discussed in Sec. V. Figure 1 plots the approximate solutions produced by the Lax-Friedrich (LxF), Lax-Wendroff (LxW), and WENO methods, as well as the exact solution for a spatial discretization using N_x computational cells. The exact solution is a shock moving to the right followed by a contact discontinuity also moving to the right, and a rarefaction wave moving to the left. Figure 1 displays a snapshot of this motion. The LxF method is seen to smooth out the shock and rarefaction wave, while the LxW method produces a more accurate solution, though it produces a nonphysical oscillation near the shock that does not vanish as N_x increases. The WENO solution is seen to be extremely accurate, matching closely the exact solution, while not generating any oscillations near the shock. Because of these properties, it is highly desirable to incorporate WENO methods into the QPDE algorithm.

In this paper we show how FV and WENO methods can be incorporated into the QPDE algorithm of Refs. [1,2]. The result is a QPDE algorithm that (i) can handle the appearance of discontinuities such as shocks, (ii) can be constructed to have arbitrarily high-order accuracy in regions where the solution is smooth, and (iii) quashes the appearance of nonphysical oscillations near discontinuities. We verify the modified QPDE algorithm on three problems of progressively increasing difficulty, and find excellent agreement between the results of numerical simulation of the QPDE algorithm and the exact solutions of these problems.

The structure of this paper is as follows. In Sec. II we briefly describe the QPDE algorithm and show how FV methods can be used to implement the spatial discretization that it requires. We then show how the WENO construction can be incorporated into the FV discretization. In Sec. III we apply the FV-WENO modified QPDE algorithm to the easiest of the three verification problems. It is an Euler gas problem with smooth initial data. The exact solution corresponds to right propagation of the initial data with unit speed. In Sec. IV we consider Sod's problem [15] which is a Riemann problem whose solution is a left-going rarefaction wave and a right-going shock followed by a right-going contact discontinuity. In Sec. V we examine a much more difficult Riemann problem due to Lax [9] whose solution is also a left-going rarefaction wave and right-going shock and contact discontinuity, though here the rarefaction wave is weaker than in Sod's problem and the shock and contact discontinuity are stronger. We make closing remarks in Sec. VI. Finally, in the Appendix, we determine the theoretical convergence order of the modified QPDE algorithm. Note that, as part of the algorithm's error analysis, we show how the quantum evaluation of integrals that lies at the heart of the QPDE algorithm can be modified to produce an algorithm that has arbitrary order of accuracy in time.

II. INCORPORATING FV AND WENO METHODS

In this Section we explain how FV and WENO methods are incorporated into the QPDE algorithm. In Sec. II A we briefly summarize the QPDE algorithm construction. For a detailed presentation of it, and the Russian-doll-like hierarchy of supporting quantum algorithms, see Ref. [16]. In Sec. II B

we show how FV methods are used to implement the spatial discretization required by the QPDE algorithm. The effect is to reduce the system of nonlinear PDEs to a coupled system of nonlinear ordinary differential equations (ODEs). Finally, in Sec. II C, we show how the WENO construction of the FV flux is incorporated into the system of nonlinear ODEs. We examine the performance of the QPDE/FV-WENO algorithm in Secs. III–V.

A. QPDE algorithm summary

The task is to find an approximate, bounded solution of a system of one-dimensional conservation laws over a time interval $0 \leqslant t \leqslant T$. Such a capability, in conjunction with dimensional splitting techniques [7,11], allows multispatial dimensional conservation laws to be solved.

We thus consider the nonlinear conservation law

$$\frac{\partial \mathbf{U}}{\partial t} + \frac{\partial}{\partial x}\mathbf{f}[\mathbf{U}] = \mathbf{0}, \tag{1}$$

subject to the initial condition $\mathbf{U}(x,0) = \mathbf{U}_0(x)$. We hold off on specifying boundary conditions until Secs. III–V where we considering specific verification problems. Here, $\mathbf{U}(x,t)$ is the exact solution which is a d-component vector field, and $\mathbf{f}[\mathbf{U}]$ is its associated nonlinear conserved flux. The verification problems will look for solutions of the Euler equations which govern the flows of an inviscid fluid [7,11,17] for which

$$\mathbf{U} = \begin{pmatrix} \rho \\ \rho u \\ E \end{pmatrix}, \quad \mathbf{f}[\mathbf{U}] = \begin{pmatrix} \rho u \\ p + \rho u^2 \\ u(E+p) \end{pmatrix}, \tag{2}$$

and ρ is the fluid density, u is the (fluid) velocity, E is the total energy density, and p is the pressure. We will assume that the fluid can be modeled as an ideal gas. The total energy density is then

$$E = \rho\left(\frac{u^2}{2} + e\right), \tag{3}$$

where the internal energy density is $e = p/(\gamma - 1)\rho$, and γ is the ratio of specific heats $\gamma = C_p/C_V$.

At its coarsest level of description, the QPDE algorithm consists of two steps. The first discretizes space while leaving time a continuous parameter. There are many ways to acomplish this. In Refs. [1–6,16], finite-difference methods were used. In Sec. II B we use finite-volume (FV) methods. The effect of the spatial discretization is to reduce Eq. (1) to a coupled system of nonlinear ODEs:

$$\frac{d\mathbf{U}_I}{dt} = \mathbf{f}_I[\mathbf{U}]. \tag{4}$$

In the FV approach, I labels the computational cells, and there is thus a system of nonlinear ODEs associated with each cell. Further discussion of the driver function $\mathbf{f}_I[\mathbf{U}]$ appears below. The second step applies a quantum ODE algorithm to obtain an approximate solution of Eq. (1). As in our previous work [1–6,16], we use Kacewicz' quantum ODE algorithm [18]. His algorithm is implemented in five steps. To keep the notation simple, we suppress the I subscript in the remainder of this section.

(1) Partition $0 \leqslant t \leqslant T$ into n primary subintervals using $n+1$ uniformly spaced intermediate times $t_0 = 0, \ldots, t_i = ih, \ldots, t_n = T$, where $h = T/n$. Let T_i denote the ith primary subinterval $[t_i, t_{i+1}]$.

(2) Partition each primary subinterval $T_i = [t_i, t_{i+1}]$ into $N_k = n^{k-1}$ secondary subintervals by introducing $N_k + 1$ uniformly spaced intermediate times $t_{i,j} = t_i + j\bar{h}$, where $j = 0, \ldots, N_k$ and $\bar{h} = h/N_k$. Let $T_{i,j}$ denote the jth secondary subinterval $[t_{i,j}, t_{i,j+1}]$ in the ith primary subinterval.

(3) Associate with each primary subinterval T_i the parameter $\mathbf{y}_i$. The parameter $\mathbf{y}_0$ is set equal to the ODE initial condition $\mathbf{y}_0 \equiv \mathbf{U}_0$. The remaining parameters $\mathbf{y}_1, \ldots, \mathbf{y}_{n-1}$ are determined in Step 5 below.

(4) Within each primary subinterval T_i, Taylor's method [19,20] is used to construct an approximate solution $\boldsymbol{\alpha}_{i,j}(t)$ for each secondary subinterval $T_{i,j}$. The approximate solution for primary subinterval T_i is then defined as $\boldsymbol{\alpha}_i(t) = \boldsymbol{\alpha}_{i,j}(t)$ when $t \in T_{i,j}$. The approximate solution $\boldsymbol{\alpha}_i(t)$ is required to be continuous throughout T_i, and its value at $t = t_i$ is defined to be $\boldsymbol{\alpha}_i(t_i) \equiv \mathbf{y}_i$.

(5) Kacewicz derives the relation

$$\mathbf{y}_{i+1} = \mathbf{y}_i + \int_{t_i}^{t_{i+1}} dt\, \mathbf{f}[\boldsymbol{\alpha}_i(t)] \quad (0 \leqslant i \leqslant n-2), \tag{5}$$

which allows the parameters $\mathbf{y}_1, \ldots, \mathbf{y}_{n-1}$ to be determined iteratively. Each iteration step requires approximate evaluation of the integral appearing in Eq. (5). Kacewicz approximates the integral using a Riemann sum and then uses the quantum amplitude estimation algorithm [21] (QAEA) to approximate its value. Note that this is the only step in the algorithm that requires a quantum computer. All other calculations are done on a classical computer. The iteration procedure begins with $\mathbf{y}_0 = \mathbf{U}_0$. Knowing $\mathbf{y}_0$ determines the approximate solution $\boldsymbol{\alpha}_0(t)$ (see Refs. [1,2,16,18] for details). Inserting $\boldsymbol{\alpha}_0(t)$ into the driver function $\mathbf{f}[\mathbf{U}]$ determines the integrand in Eq. (5) for $i = 0$. The integral is approximated as just explained and the value returned by the QAEA is (classically) added to $\mathbf{y}_0$ to give $\mathbf{y}_1$. Knowing $\mathbf{y}_1$ then determines $\boldsymbol{\alpha}_1(t)$, which determines the integrand $\mathbf{f}[\boldsymbol{\alpha}_1(t)]$. The QAEA is used to return an approximate value for the integral in Eq. (5) for $i = 1$, which is then added to $\mathbf{y}_1$ to give $\mathbf{y}_2$, etc. At the conclusion of the iteration procedure all approximate solutions $\boldsymbol{\alpha}_0(t), \ldots, \boldsymbol{\alpha}_{n-1}(t)$ have been determined, and the approximate ODE solution is then $\boldsymbol{\alpha}(t) = \boldsymbol{\alpha}_i(t)$ for $t \in T_i$. Kacewicz [18] showed that his algorithm provides a quadratic speedup over classical nonlinear ODE algorithms and Ref. [2] showed that the QPDE algorithm inherits this speedup.

Remark 1. Implementing Kacewicz's algorithm on an actual quantum computer requires a quantum integration algorithm, such as Ref. [22], to pass the integrand values appearing in the Riemann sum to the quantum computer. Once these values have been passed, the quantum computer would run the QAEA to determine an approximate value of the Riemann sum. Reference [16] shows how this hierarchy of quantum algorithms can be implemented with quantum circuits.

Remark 2. As noted above, Kacewicz's algorithm approximates an integral by a Riemann sum which gives a first-order accurate result. Higher-order accuracy is possible with much fewer function evaluations if Gaussian quadrature [19] is used to approximate the integral. We show how this can be done in the Appendix. We use this modified version of Kacewicz's

algorithm in the numerical simulations presented in Secs. III–V, using fourth-order accurate Gaussian quadrature.

B. Spatial discretization through FV methods

Suppose the spatial domain for the PDE solution is $x_{\min} \leqslant x \leqslant x_{\max}$. We introduce $M+1$ gridpoints $x_i = x_{\min} + i\Delta x$, with $i = 0, \ldots, M$, where $\Delta x = (x_{\max} - x_{\min})/M$. We define the computational cell centered on x_i to be $C_i = [x_{i-1/2}, x_{i+1/2}]$, where $x_{i\pm1/2} = x_i \pm \Delta x/2$ are the cell boundaries. For simplicity, we assume that Δx is constant. Lastly, we introduce the cell average of the conserved variables $\mathbf{U}(x,t)$:

$$\bar{\mathbf{U}}_i(t) = \frac{1}{\Delta x}\int_{x_{i-1/2}}^{x_{i+1/2}} dx\, \mathbf{U}(x,t). \tag{6}$$

The cell averages $\{\bar{\mathbf{U}}_i(t)\}$ are the unknowns to be determined. To handle boundary conditions in the WENO scheme it is necessary to introduce ghost cells on each side of the spatial domain. For the fifth-order WENO scheme used in the simulations in this paper, three ghost cells are needed per side. For a discussion of boundary conditions and ghost cells, see Ref. [7], Chap. 7.

To determine the equations governing the time evolution of the cell averages we integrate Eq. (1) over the computational cell C_i and apply Gauss' law to the flux term to obtain

$$\frac{d\bar{\mathbf{U}}_i}{dt} + \frac{1}{\Delta x}\{\mathbf{f}[\mathbf{U}_{i+1/2}] - \mathbf{f}[\mathbf{U}_{i-1/2}]\} = \mathbf{0}. \tag{7}$$

Upon expressing the cell interface values $\mathbf{U}_{i\pm1/2}$ in terms of the cell averages $\bar{\mathbf{U}}_i$ (see Sec. II C) we see that the system of PDEs has been converted to a coupled set of ODEs, one for each cell C_i, thus implementing the first step of the QPDE algorithm.

In general, Eq. (7) must be solved numerically, and linear stability requires introducing a numerical flux $\hat{\mathbf{f}}[\mathbf{U}^-, \mathbf{U}^+]$ that propagates the solution along characteristics. This requires the numerical flux to satisfy the following conditions [14]: (i) $\hat{\mathbf{f}}[\mathbf{U}^-, \mathbf{U}^+]$ is nondecreasing (nonincreasing) in $\mathbf{U}^-$ ($\mathbf{U}^+$); (ii) $\hat{\mathbf{f}}[\mathbf{U}, \mathbf{U}] = \mathbf{f}[\mathbf{U}]$; and (iii) $\hat{\mathbf{f}}[\mathbf{U}^-, \mathbf{U}^+]$ is Lipschitz continuous in each of its arguments. In this paper we use the Lax-Friedrichs flux,

$$\hat{\mathbf{f}}[\mathbf{U}^-, \mathbf{U}^+] = \tfrac{1}{2}\{\mathbf{f}[\mathbf{U}^-] + \mathbf{f}[\mathbf{U}^+] - \theta(\mathbf{U}^+ - \mathbf{U}^-)\}, \tag{8}$$

where θ is the absolute maximum characteristic velocity, which is determined from the components of the $\mathbf{U}_i$ according to $\theta = \max_i [|\bar{U}_i| + \sqrt{\gamma p_i/\rho_i}]$. Equation (7) then becomes

$$\frac{d\bar{\mathbf{U}}_i}{dt} = \frac{1}{\Delta x}\{\hat{\mathbf{f}}[\mathbf{U}^-_{i-1/2}, \mathbf{U}^+_{i-1/2}] - \hat{\mathbf{f}}[\mathbf{U}^-_{i+1/2}, \mathbf{U}^+_{i+1/2}]\}. \tag{9}$$

Section II C explains how $\mathbf{U}^-_{i\pm1/2}$ and $\mathbf{U}^+_{i\pm1/2}$ are reconstructed from the cell averages $\{\bar{\mathbf{U}}_i\}$ in a WENO scheme. Once they have been determined and inserted into Eq. (9), the right-hand side becomes the driver function $\mathbf{f}_i[\bar{\mathbf{U}}]$, and the system of nonlinear ODEs to be solved with a quantum computer (as described in Sec. II A) is

$$\frac{d\bar{\mathbf{U}}_i}{dt} = \mathbf{f}_i[\bar{\mathbf{U}}]. \tag{10}$$

C. Incorporating WENO reconstruction

Although WENO reconstruction methods allow for arbitrarily large order of convergence, in this paper we focus on the fifth-order accurate reconstruction of $\mathbf{U}^{\pm}_{i\pm1/2}$ [14]. We illustrate the reconstruction for a scalar function $\phi(x,t)$ with cell averages $\bar{\phi}_i(t)$. The $\bar{\phi}_i$ should be thought of as one of the components of the cell average $\bar{\mathbf{U}}_i$.

(i) For a fifth-order method, $\phi^{\pm}_{i\pm1/2}$ are determined using a five-element stencil. For $\phi^-_{i+1/2}$, the stencil consists of the cells with labels $\{i-2, i-1, i, i+1, i+2\}$:

$$\begin{aligned}\phi^-_{i+1/2} = {} & w_1\big[\tfrac{1}{3}\bar{\phi}_{i-2} - \tfrac{7}{6}\bar{\phi}_{i-1} + \tfrac{11}{6}\bar{\phi}_i\big] \\ & + w_2\big[-\tfrac{1}{6}\bar{\phi}_{i-1} + \tfrac{5}{6}\bar{\phi}_i + \tfrac{1}{3}\bar{\phi}_{i+1}\big] \\ & + w_3\big[\tfrac{1}{3}\bar{\phi}_i + \tfrac{5}{6}\bar{\phi}_{i+1} - \tfrac{1}{6}\bar{\phi}_{i+2}\big].\end{aligned} \tag{11}$$

Here, $\{w_j | j \in \{1,2,3\}\}$ are the nonlinear weights

$$w_j = \frac{\tilde{w}_j}{\sum_j \tilde{w}_j}, \quad \tilde{w}_j = \frac{\gamma_j}{(\epsilon_w + \beta_j)^2}, \tag{12}$$

where the linear weights are $\gamma_1 = 1/10$, $\gamma_2 = 3/5$, $\gamma_3 = 3/10$, and $\epsilon_w = 10^{-6}$. The smoothness indicators $\{\beta_j | j \in \{1,2,3\}\}$ are

$$\begin{aligned}\beta_1 &= \tfrac{13}{12}(\bar{\phi}_{i-2} - 2\bar{\phi}_{i-1} + \bar{\phi}_i)^2 + \tfrac{1}{4}(\bar{\phi}_{i-2} - 4\bar{\phi}_{i-1} + 3\bar{\phi}_i)^2, \\ \beta_2 &= \tfrac{13}{12}(\bar{\phi}_{i-1} - 2\bar{\phi}_i + \bar{\phi}_{i+1})^2 + \tfrac{1}{4}(\bar{\phi}_{i-1} - \bar{\phi}_{i+1})^2, \\ \beta_3 &= \tfrac{13}{12}(\bar{\phi}_i - 2\bar{\phi}_{i+1} + \bar{\phi}_{i+2})^2 + \tfrac{1}{4}(3\bar{\phi}_i - 4\bar{\phi}_{i+1} + \bar{\phi}_{i+2})^2.\end{aligned} \tag{13}$$

Note that $\phi^+_{i+1/2}$ can be obtained from Eq. (11) by mirror reflecting the stencil labels about the point $x_{i+1/2}$. Thus $\bar{\phi}_{i-k} \to \bar{\phi}_{i+1+k}$ in Eq. (11). Specifically, $\bar{\phi}_{i-2} \to \bar{\phi}_{i+3}, \ldots, \bar{\phi}_{i+2} \to \bar{\phi}_{i-1}$, and so the stencil is now $\{i-1, i, i+1, i+2, i+3\}$. Finally, to obtain $\phi^{\pm}_{i-1/2}$, let $i \to i-1$ in the formulas for $\phi^{\pm}_{i+1/2}$, respectively. The stencil for $\phi^-_{i-1/2}$ is now $\{i-3, i-2, i-1, i, i+1\}$, and for $\phi^+_{i-1/2}$ is $\{i-2, i-1, i, i+1, i+2\}$.

(ii) As pointed out in Ref. [13], the above scheme works well up to third-order accuracy. However, for higher-order accuracy, the WENO reconstruction procedure is best carried out using local characteristic variables. Their construction involves local diagonalization of the physical flux Jacobian $A = \partial\mathbf{f}/\partial\mathbf{U}$. For the Euler equations for an ideal gas [11] with gas velocity u, (gas) mass density ρ, and total energy density E, A has real eigenvalues $\{u-a, u, u+a\}$, where $a = \sqrt{\gamma p/\rho}$ is the local speed of sound; $p = (\gamma-1)(E - \rho u^2/2)$ is the pressure; and $\gamma = 1.4$ for air under standard conditions. A basis for the left and right eigenspaces of A is formed, respectively, by the rows of R^{-1} and the columns of R:

$$R^{-1} = \frac{1}{2h}\begin{pmatrix}[u^2/2 + uh/a] & -[h/a + u] & 1 \\ [2h - u^2] & 2u & -2 \\ [u^2/2 - uh/a] & [h/a - u] & 1\end{pmatrix}, \tag{14}$$

$$R = \begin{pmatrix}1 & 1 & 1 \\ [u-a] & u & [u+a] \\ [H - ua] & u^2/2 & [H + ua]\end{pmatrix}. \tag{15}$$

Here, $h = a^2/(\gamma-1)$ is the specific enthalpy and $H = h + u^2/2$ is the total specific enthalpy.

The local characteristic variables $\bar{\mathbf{V}}_i$ are related to the conserved variables $\bar{\mathbf{U}}_i$ through the relations

$$\bar{\mathbf{V}}_i = \tilde{R}^{-1}\bar{\mathbf{U}}_i, \tag{16}$$

$$\bar{\mathbf{U}}_i = \tilde{R}\bar{\mathbf{V}}_i, \tag{17}$$

where $\tilde{R}^{-1}$ and $\tilde{R}$ are determined by Eqs. (14) and (15) evaluated at the Roe-average velocity $\tilde{u}$, total specific enthalpy $\tilde{H}$, specific enthalpy $\tilde{h}$, and local speed of sound $\tilde{a}$:

$$\tilde{u} = \frac{\sqrt{\rho_i}\bar{u}_i + \sqrt{\rho_{i+1}}\bar{u}_{i+1}}{\sqrt{\rho_i} + \sqrt{\rho_{i+1}}}, \tag{18}$$

$$\tilde{H} = \frac{\sqrt{\rho_i}\bar{H}_i + \sqrt{\rho_{i+1}}\bar{H}_{i+1}}{\sqrt{\rho_i} + \sqrt{\rho_{i+1}}}, \tag{19}$$

and

$$\tilde{h} = \tilde{H} - \frac{1}{2}\tilde{u}^2, \tag{20}$$

$$\tilde{a} = \sqrt{(\gamma - 1)\tilde{h}}. \tag{21}$$

We see from Eqs. (18)–(21) that the tilde variables are determined by the cell averages of the primitive variables (ρ, u, E) at cells i and $i + 1$.

Having $\tilde{R}^{-1}$, the characteristic variables $\bar{\mathbf{V}}_i$ are determined from the conserved variables $\bar{\mathbf{U}}_i$ through Eq. (16). The $\bar{\mathbf{V}}_i$ are then used to reconstruct $\mathbf{V}^{\pm}_{i\pm1/2}$ using the above WENO reconstruction on each of its components. Having the $\mathbf{V}^{\pm}_{i\pm1/2}$ in hand, we use Eq. (17) to obtain $\mathbf{U}^{\pm}_{i\pm1/2}$, which are then inserted into the Lax-Friedrichs flux [Eq. (8)] to obtain the driver function $\mathbf{f}_i[\bar{\mathbf{U}}]$ appearing in Eq. (10) via Eq. (9).

III. VERIFICATION PROBLEM 1: SMOOTH SOLUTION

In this Section we use the QPDE/FV-WENO algorithm to obtain an approximate solution to the Euler equations, subject to a smooth initial condition and periodic boundary conditions. The exact solution is known analytically, and is used to test the quality of the QPDE/FV-WENO approximate solution. This problem, as well as those considered in Secs. IV and V, are taken from Ref. [23]. In Sec. III A we (i) state the mixed initial-boundary value problem to be solved, (ii) give its exact solution, and (iii) define the error and convergence order of the numerical solution. In Sec. III B we present the numerical simulation results obtained by applying the QPDE/FV-WENO algorithm to this problem. We determine the error and convergence order of the approximate solution, and compare the latter to the theoretical convergence order determined in the Appendix.

The following parameter values were used in the numerical simulations presented in this Section: (i) Courant-Friedrichs-Lewy (CFL) constant $C_0 = 0.075$ (0.6) for the quantum (classical) simulations; (ii) ratio of specific heats $\gamma = 1.4$; (iii) quantum amplitude estimation algorithm [16,21] error upper bound $\epsilon_1 = 5 \times 10^{-3}$; and (iv) probability $\delta = 1 \times 10^{-3}$ that the approximate solution error fails to satisfy the upper bound derived by Kacewicz [18]. Finally, the Supplemental Material for Ref. [1] explains how values are assigned to the parameters n and k associated with the partitioning of time into primary and secondary subintervals.

Note that because the Euler equations are covariant under a change of units, the axes of all figures in this paper are in arbitrary units. Thus if a point on a plot has $x = 1$, this means $x = 1$ m in mks units, 1 cm in cgs units, and similarly for other physical quantities.

A. Problem statement

The task is to determine an approximate solution to the Euler equations [Eqs. (1) and (2)] for $0 \leqslant t \leqslant T = 2$ and $0 \leqslant x \leqslant 2$, subject to periodic boundary conditions, and the smooth initial condition $(\rho, u, p) = (1 + 0.2\sin\pi x, 1, 1)$. The fluid is assumed to be an ideal gas as described in Sec. II A.

The exact solution for this problem is

$$[\rho(x,t), u(x,t), p(x,t)] = [1 + 0.2\sin\pi(x - t), 1, 1]. \tag{22}$$

To verify this statement, first insert the proposed solutions for u and p into Eq. (3). This gives $E = \rho/2 + 2.5$. Next, plug in E, u, and p into the Euler equations. The result is that all three Euler PDEs reduce to the same advection equation,

$$\frac{\partial\rho}{\partial t} + \frac{\partial\rho}{\partial x} = 0, \tag{23}$$

which by inspection has unit advection velocity in the $+x$ direction. As is well known [7], the exact solution is advection of the initial condition for ρ in the $+x$ direction with unit velocity, $\rho(x,t) = 1 + 0.2\sin\pi(x - t)$. Thus, Eq. (22) is the exact solution.

To test the quality of the QPDE/FV-WENO approximate solution we define the error $\varepsilon_{n,M}$ for $\rho(x,T)$ as

$$\varepsilon_{n,M} = \|\rho_{\rm ex}(x,T) - \rho_{\rm QPDE}(x,T)\|_n, \tag{24}$$

with similar definitions for $\rho(x,t)u(x,t)$ and $E(x,t)$. Here, the exact solution is found using an exact Riemann solver [11]; M is the number of FV cells; and for an arbitrary function $F(x,T)$, $\|F(x,T)\|_n$ denotes the L_n norm,

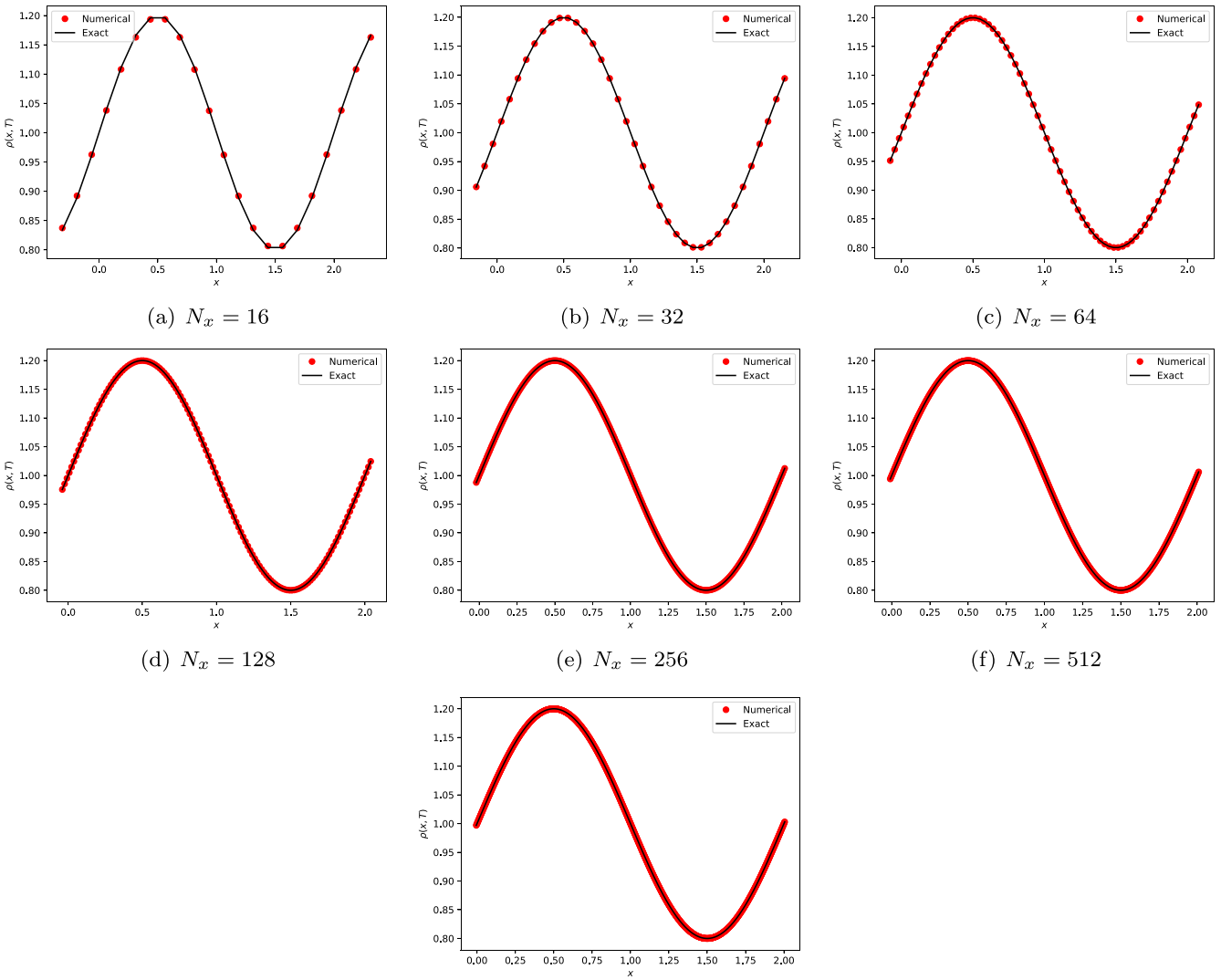
$$\|F(x,T)\|_n = \left(\sum_{i=1}^{M} |F(x_i,T)|^n \Delta x\right)^{1/n}, \tag{25}$$

where Δx is the size of a FV cell. Note that in our calculation of the error, we used the cell averages $\{\bar{F}_i(T)\}$ rather than $\{F(x_i,T)\}$ as it is the cell averages that are found by the exact Riemann solver and the QPDE simulation. In Sec. III B we present error results based on the L_1, L_2, and L_∞ norms. The latter norm is given by $|F(x_{i_*},T)|$, where i_* labels the gridpoint x_i for which $|F(x_i,T)|$ is largest. We will evaluate these errors for numerical simulations using $M \equiv N_x = 16, 32, \ldots, 512, 1024$ FV cells. Finally, we define the *numerical* convergence order O^c_{n,N_x} as

$$O^c_{n,N_x} = \frac{\ln\left(\frac{\varepsilon_{n,N_x}}{\varepsilon_{n,N_x/2}}\right)}{\ln 2}. \tag{26}$$

B. Simulation results

Here, we present the numerical simulation results generated by simulating the application of the QPDE/FV-WENO algorithm to the Euler equations problem introduced in



(a) $N_x = 16$ (b) $N_x = 32$ (c) $N_x = 64$

(d) $N_x = 128$ (e) $N_x = 256$ (f) $N_x = 512$

(g) $N_x = 1024$

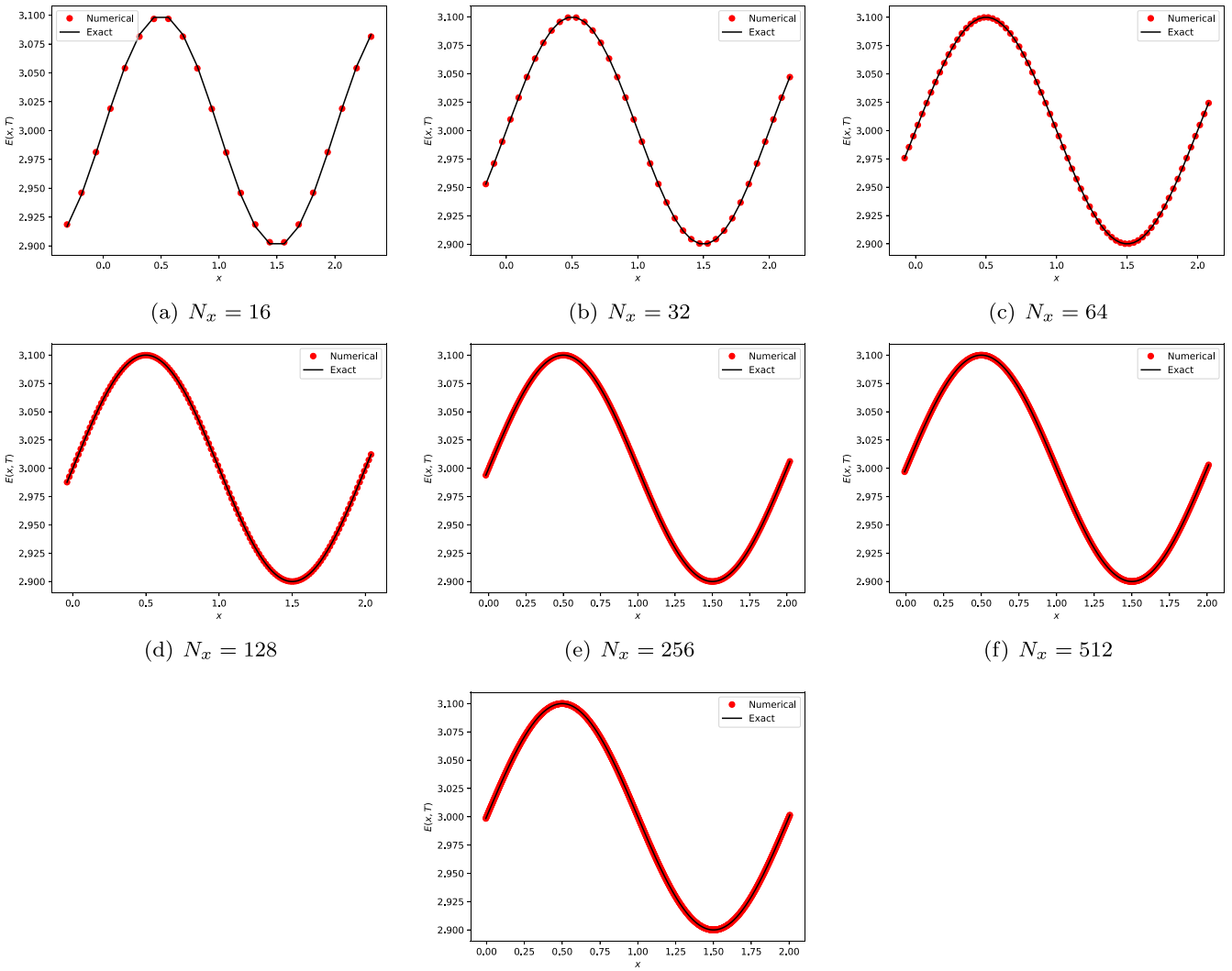
FIG. 2. Comparing QPDE/FV-WENO simulation results for mass density against the exact solution for verification problem 1. (a)–(g) plot the mass density $\rho(x, T = 2)$ for spatial discretizations using $N_x = 16-1024$ FV computational cells, respectively. The quantum simulation results are plotted as red dots, while the exact solution corresponds to the black curve.

Sec. III A. We compare our quantum results with the exact solution, as well as to a classical WENO algorithm that uses the fourth-order Runge-Kutta (RK4) algorithm [13] to do its time stepping. Both the QPDE/FV-WENO algorithm and the classical WENO/RK4 algorithm use the WENO reconstruction procedure which is fifth-order accurate in the size Δx of a FV cell (see Sec. II C).

Before presenting our results, recall that the exact solution has $u(x, t) = 1$ and so $\rho(x, t)$ and $\rho(x, t)u(x, t)$ are identical. This is true to high accuracy in our simulation results and so we only show results for $\rho(x, T)$ and $E(x, T)$, where $T = 2$.

Figures 2 and 3 compare the QPDE/FV-WENO simulation results with the exact solution for $\rho(x, T)$ and $E(x, T)$, respectively. The quantum results were calculated for $N_x = 16, \ldots, 1024$ FV cells. We see that the quantum results lie on the exact solution curves for all values of N_x. At this crudest level of comparison, the quantum solution is fully consistent with the exact solution.

In Tables I and II we present results for the numerical errors ε_{n,N_x} [Eq. (24)] and numerical convergence order O^c_{n,N_x} [Eq. (26)] for $\rho(x, T)$ for the QPDE/FV-WENO algorithm and the classical WENO/RK4 algorithm, respectively. As noted above, the errors are determined using the L_1, L_2, and L_∞ norms [Eq. (25)]. Similarly, in Tables III and IV, we present the numerical error and convergence order for $E(x, T)$ for the QPDE/FV-WENO and classical WENO/RK4 solutions, respectively. We see that (i) the QPDE/FV-WENO error results for both $\rho(x, T)$ and $E(x, T)$ are of order 10^{-3} for $N_x = 16$, and 10^{-12} for $N_x = 1024$, and (ii) the convergence order ranges from 4.6 to 4.8 for $N_x = 32$, and from 4.8 to 5.4 for $N_x = 1024$ for both $\rho(x, T)$ and $E(x, T)$, depending on the norm. Note that in all cases, the smaller convergence orders resulted from the L_∞ norm. Comparing these quantum results against the classical WENO/RK4 results, we see the QPDE/FV-WENO algorithm delivers the same level of performance as the workhorse classical WENO/RK4 algorithm.



(a) $N_x = 16$ (b) $N_x = 32$ (c) $N_x = 64$

(d) $N_x = 128$ (e) $N_x = 256$ (f) $N_x = 512$

(g) $N_x = 1024$

FIG. 3. Comparing QPDE/FV-WENO simulation results for total energy density against the exact solution for verification problem 1. (a)–(g) plot the total energy density $E(x, T = 2)$ for spatial discretizations using $N_x = 16$–1024 FV computational cells, respectively. The quantum simulation results are plotted as red dots, while the exact solution corresponds to the black curve.

In the Appendix we noted that the QPDE/FV-WENO algorithm has three sources of error due to: (i) the WENO reconstruction procedure; (ii) replacing the integrals arising in Kacewicz's quantum ODE algorithm by a discrete sum; and (iii) trunctating the Taylor series representation of the solution to $r + 1$ terms. Table V displays the scaling relation (derived in the Appendix) for each of these errors. It was shown in Appendix 4 that for the simulations reported in this paper, the discretization error $\varepsilon_d = \mathcal{O}(\bar{h}^4)$ is the dominant error. The resulting theoretical convergence order was shown to be 4. From Tables I and III we see that the QPDE/FV-WENO algorithm has a much better numerical convergence order, in fact,

TABLE I. QPDE/FV-WENO simulation error and convergence order for $\rho(x, T = 2)$.

N_x	ε_{1,N_x}	O^c_{1,N_x}	ε_{2,N_x}	O^c_{2,N_x}	ε_{∞,N_x}	O^c_{∞,N_x}
16	3.544×10^{-3}		2.665×10^{-3}		2.535×10^{-3}	
32	1.268×10^{-4}	4.803	1.013×10^{-4}	4.716	1.180×10^{-4}	4.552
64	4.099×10^{-6}	4.951	3.461×10^{-6}	4.872	4.952×10^{-6}	4.447
128	1.225×10^{-7}	5.064	9.759×10^{-8}	5.148	1.127×10^{-7}	5.457
256	3.730×10^{-9}	5.037	2.974×10^{-9}	5.035	3.198×10^{-9}	5.139
512	1.062×10^{-10}	5.134	8.426×10^{-11}	5.141	8.849×10^{-11}	5.175
1024	2.596×10^{-12}	5.354	2.014×10^{-12}	5.386	3.117×10^{-12}	4.827

TABLE II. Classical WENO solver with fourth-order Runge-Kutta time-stepper simulation error and convergence order for $\rho(x, T = 2)$.

N_x	ε_{1,N_x}	O^c_{1,N_x}	ε_{2,N_x}	O^c_{2,N_x}	ε_{∞,N_x}	O^c_{∞,N_x}
16	3.545×10^{-3}		2.670×10^{-3}		2.557×10^{-3}	
32	1.269×10^{-4}	4.803	1.015×10^{-4}	4.717	1.082×10^{-4}	4.563
64	3.952×10^{-7}	5.005	3.168×10^{-6}	5.001	3.650×10^{-6}	4.889
128	1.225×10^{-7}	5.010	9.769×10^{-8}	5.019	1.126×10^{-7}	5.018
256	3.733×10^{-9}	5.037	2.977×10^{-9}	5.035	3.206×10^{-9}	5.134
512	1.060×10^{-10}	5.138	8.414×10^{-11}	5.145	8.862×10^{-11}	5.177
1024	2.482×10^{-12}	5.416	1.944×10^{-12}	5.430	2.037×10^{-12}	5.443

more like of $5.x$. This is also true of the classical WENO/RK4 algorithm which has a theoretical convergence order of 4 due to the fourth-order Runge-Kutta time stepper, but as seen from Tables II and IV, a numerical convergence order more as $5.x$. Both the quantum and classical algorithms deliver better than expected convergence for the problem considered here which has a smooth, simple solution (a sine wave). We see that for smooth solutions, just as with the classical WENO algorithm, the QPDE/FV-WENO algorithm delivers an approximate solution with high-order accuracy and convergence.

IV. VERIFICATION PROBLEM 2: SOD'S PROBLEM

In this Section and the next, we apply the QPDE/FV-WENO algorithm to *Riemann problems*. Here. we consider a Riemann problem introduced by Sod [15].

Sod's problem looks for a solution of the Euler equations subject to (i) outflow boundary conditions (implemented using zero-order extrapolation [24]), and (ii) initial condition

$$(\rho, u, p) = \begin{cases} (1, 0, 1) & (x < 0), \\ (0.125, 0, 0.1) & (x > 0), \end{cases} \tag{27}$$

that is *discontinuous* at $x = 0$. For the numerical simulations discussed below, we chose $-5 \leqslant x \leqslant 5$ and $0 \leqslant t \leqslant T = 0.16$. We used an exact Riemann solver [11] to obtain the exact solution which consists of a left-going rarefaction wave and a right-going shock wave followed by a right-going contact discontinuity. The shock wave and contact discontinuity correspond to locations where the exact solution is discontinuous, while the rarefaction wave, although continuous, is bracketed by locations where the first derivative (of the exact solution) is discontinuous. These three types of discontinuities are clearly visible in Figs. 4–6. The shock wave and contact discontinuity are of modest size, while the rarefaction wave is stronger. Note that the numerical convergence order is typically not calculated in Riemann problems as the error near discontinuities dominates the error in regions where the solution is smooth, and so the numerical convergence order will be less than for a smooth solution.

In Fig. 4 we plot the simulation results for the mass density $\rho(x, T = 0.16)$ obtained using (i) the QPDE/FV-WENO algorithm, and (ii) the exact solution. In these simulations, the parameters γ, ϵ_1, and δ introduced in Sec. III take on the values given there, while the CFL parameter is now $C_0 = 0.0375$. As in Sec. III, we show results for $N_x = 16, 32, \ldots, 1024$ FV cells. We see that for $N_x = 16$ and 32, the quantum results follow the shape of the exact solution, though all three discontinuity types are smeared out. For $N_x = 64$, 128, and 256, the quantum results have captured the rarefaction and shock waves, though there is still a small smearing of the contact discontinuity. Finally, for $N_x = 512$ and 1024, the quantum solution has succeeded in capturing all three discontinuities. Note that, in all cases, *no oscillations* appear in the quantum results near the discontinuities in the exact solution. WENO has thus delivered its signature effect to the QPDE algorithm.

In Figs. 5 and 6 we plot our simulation results for the remaining conserved variables $\rho(x, T)u(x, T)$ and $E(x, T)$, respectively. All remarks made for Fig. 4 are also valid for Figs. 5 and 6.

We see that the QPDE/FV-WENO algorithm has succeeded in solving Sod's Riemann problem. By incorporating FV methods into the quantum algorithm, it has succeeded in finding the exact solution even though it contains moving discontinuities. Furthermore, incorporation of WENO methods into the QPDE algorithm has insured that nonphysical oscillations were *not* generated near discontinuities.

V. VERIFICATION PROBLEM 3: A PROBLEM DUE TO LAX

Here, we consider a second Riemann problem, this time due to Lax [9]. As with Sod's problem, we look for a solution

TABLE III. QPDE/FV-WENO simulation error and convergence order for $E(x, T = 2)$.

N_x	ε_{1,N_x}	O^c_{1,N_x}	ε_{2,N_x}	O^c_{2,N_x}	ε_{∞,N_x}	O^c_{∞,N_x}
16	1.772×10^{-3}		1.332×10^{-3}		1.267×10^{-3}	
32	6.344×10^{-5}	4.803	5.069×10^{-5}	4.716	5.402×10^{-5}	4.552
64	2.049×10^{-6}	4.951	1.730×10^{-6}	4.872	2.476×10^{-6}	4.447
128	6.126×10^{-8}	5.064	4.879×10^{-8}	5.148	5.637×10^{-8}	5.457
256	1.865×10^{-9}	5.037	1.487×10^{-9}	5.035	1.599×10^{-9}	5.139
512	5.313×10^{-11}	5.133	4.215×10^{-11}	5.140	4.426×10^{-11}	5.174
1024	1.302×10^{-12}	5.350	1.011×10^{-12}	5.381	1.590×10^{-12}	4.798

TABLE IV. Classical WENO solver with fourth-order Runge-Kutta time-stepper simulation error and convergence order for $E(x, T = 2)$.

N_x	ε_{1,N_x}	O^c_{1,N_x}	ε_{2,N_x}	O^c_{2,N_x}	ε_{∞,N_x}	O^c_{∞,N_x}
16	1.772×10^{-3}		1.335×10^{-3}		1.278×10^{-3}	
32	6.349×10^{-5}	4.803	5.075×10^{-5}	4.717	5.410×10^{-5}	4.563
64	1.976×10^{-6}	5.005	1.584×10^{-6}	5.001	1.825×10^{-6}	4.889
128	6.131×10^{-8}	5.010	4.884×10^{-8}	5.019	5.631×10^{-8}	5.018
256	1.866×10^{-9}	5.037	1.488×10^{-9}	5.035	1.603×10^{-9}	5.134
512	5.300×10^{-11}	5.138	4.207×10^{-11}	5.145	4.431×10^{-11}	5.176
1024	1.241×10^{-12}	5.415	9.734×10^{-13}	5.433	1.029×10^{-12}	5.427

of the Euler equations subject to outflow boundary conditions (zero-order extrapolation [24]), though with a different initial condition,

$$(\rho, u, p) = \begin{cases} (0.445, 0.698, 3.528) & (x < 0), \\ (0.5, 0, 0.571) & (x > 0), \end{cases} \tag{28}$$

that is *discontinuous* at $x = 0$. The computational domain is again $-5 \leqslant x \leqslant 5$ and $0 \leqslant t \leqslant T = 0.16$. The exact solution is found using an exact Riemann solver [11], and consists of a left-going rarefaction wave, and a right-going shock wave followed by a contact discontinuity. In Lax's problem, however, the shock wave and contact discontinuity are strong, while the rarefaction wave is weak. As explained in Sec. IV, the numerical convergence order is typically not evaluated in Riemann problems.

In Fig. 7 we plot the simulation results for the mass density $\rho(x, T = 0.16)$ obtained using (i) the QPDE/FV-WENO algorithm, and (ii) the exact solution. In these simulations, the parameters γ, ϵ_1, and δ introduced in Sec. III take on the values given there, while the CFL parameter is now $C_0 = 0.0375$. As in Secs. III and IV, we show results for $N_x = 16, \ldots, 1024$ FV cells. For $N_x = 16$ and 32, we see that the quantum results follow the shape of the exact solution, although there is substantial discrepancy for the shock wave and contact discontinuity. The quantum results do better with the rarefaction wave, although a slight amount of smearing is apparent. For $N_x = 64$, 128, and 256 (i) the rarefaction wave has been captured, (ii) capture of the shock wave gets better and better as N_x increases, and (iii) the contact discontinuity is, for the most part, captured, though a slight amount of smearing is visible. Finally, for $N_x = 512$ and 1024, all three discontinuities have been captured. As with Sod's problem, in all cases, WENO has succeeded in quashing the appearance of non-physical oscillations near discontinuities.

In Figs. 8 and 9, we plot our simulaion results for the remaining conserved variables $\rho(x, T)u(x, T)$ and $E(x, T)$, respectively. All remarks made for Fig. 7 carry over for Figs. 8 and 9.

We see that the QPDE/FV-WENO algorithm has succeeded in solving Lax's Riemann problem. Incorporation of FV and WENO methods has allowed the quantum algorithm to, respectively, find the exact solution even though it contains moving discontinuities, and to quash the appearance of nonphysical oscillations near discontinuities.

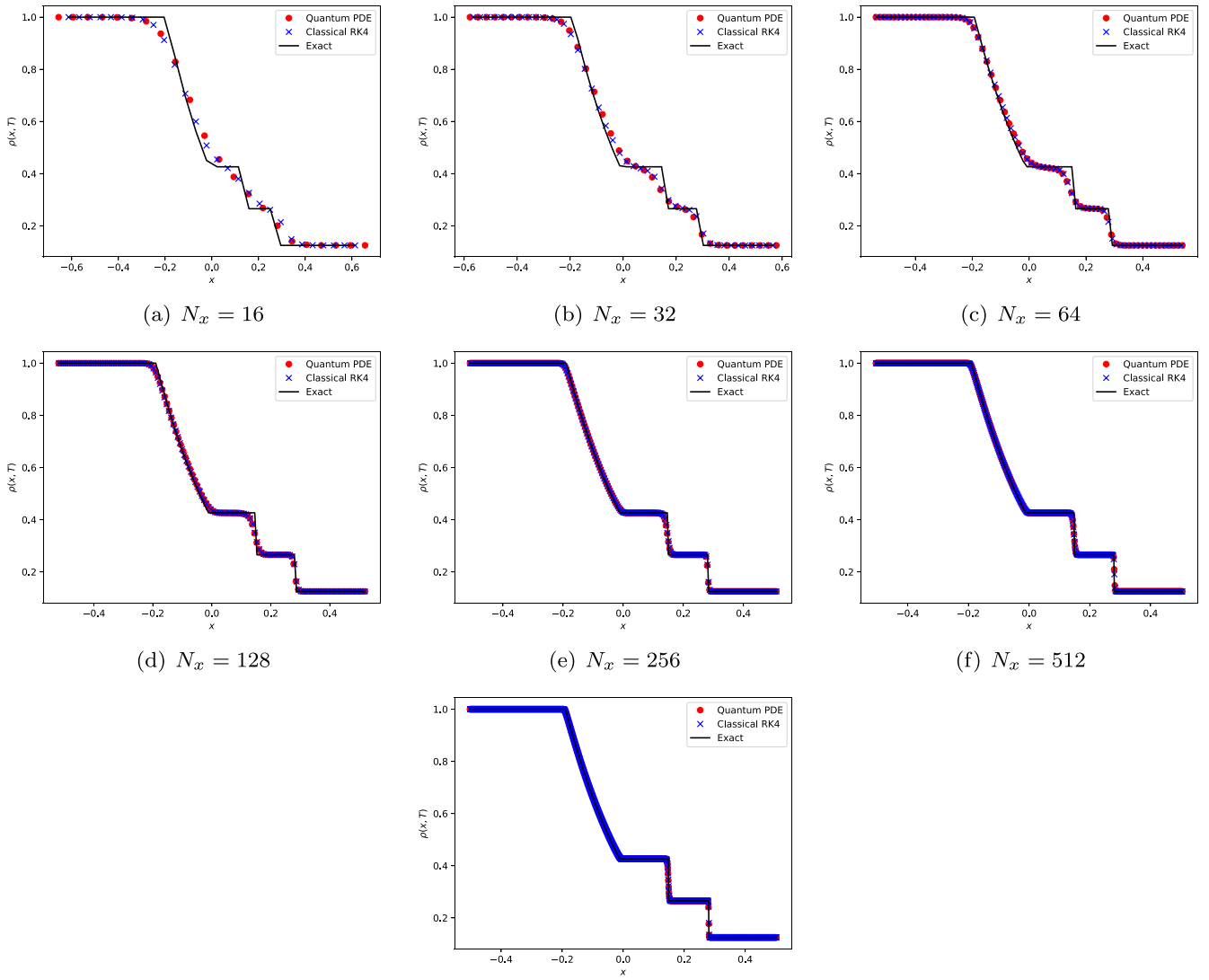
VI. DISCUSSION

In this paper we have shown how FV and WENO methods can be incorporated into the QPDE algorithm introduced in Refs. [1,2]. The introduction of FV methods allows the QPDE algorithm to handle conservation law solutions that can develop physically interesting discontinuities such as shock waves. The WENO methods allow the QPDE algorithm to produce solutions of arbitrarily high-order accuracy in regions where the solution is smooth, while quashing the appearance of nonphysical oscillations near discontinuities. We tested the QPDE/FV-WENO algorithm on three verification problems. The first problem has a smooth solution and it was shown that the quantum algorithm produced a solution with fifth-order convergence, comparable to the results produced using a classical fourth-order Runge-Kutta/WENO PDE solver. The other two problems were Riemann problems whose exact solution contained moving discontinuities. The QPDE/FV-WENO results were compared to the exact solution found using an exact Riemann solver, with the number of FV cells varying over a wide range: $N_x = 16, 32, \ldots, 512, 1024$. The agreement became better and better as N_x increased, with excellent agreement for $N_x = 512$ and 1024. Finally, we introduced a modification of Kacewicz's quantum nonlinear ODE algorithm [18], replacing his Riemann sum approximation of the integrals that are central to his algorithm with a Gaussian quadrature approximation that allows arbitrarily high-order accuracy of the integral using significantly fewer function evaluations.

The ability of the QPDE algorithm to now handle conservation law solutions with discontinuities opens up a rich

TABLE V. Error scaling relations for the sources of error in QPDE/FV-WENO algorithm. See the Appendix for their derivation; $\bar{h}$ is the duration of a secondary subinterval (see Sec. II A).

Error source	Error scaling	Impact
nth-order WENO reconstruction	$\varepsilon_w = \mathcal{O}(\bar{h}^n)$	Solution accuracy in smooth regions
nth-order Gaussian quadrature integral approximation	$\varepsilon_d = \mathcal{O}(\bar{h}^{2n})$	Accuracy of integral approximation
$(r+1)$th-order Taylor series truncation	$\varepsilon_t = \mathcal{O}(\bar{h}^{r+2})$	Solution accuracy in a secondary subinterval



(a) $N_x = 16$ (b) $N_x = 32$ (c) $N_x = 64$

(d) $N_x = 128$ (e) $N_x = 256$ (f) $N_x = 512$

(g) $N_x = 1024$

FIG. 4. Comparing QPDE/FV-WENO simulation results for mass density against the exact solution for verification problem 2: Sod's problem. (a)–(g) plot the mass density $\rho(x, T = 0.16)$ for spatial discretizations using $N_x = 16$–1024 FV computational cells, respectively. The quantum simulation results are plotted as red dots, while the exact solution corresponds to the black curve.

new application area for it in which shock waves are present and can interact with each other, and with surfaces. A few important applications where these situations can arise are supersonic/hypersonic fluid dynamics, plasma dynamics, radiation hydrodynamics, and magnetohydrodynamics.

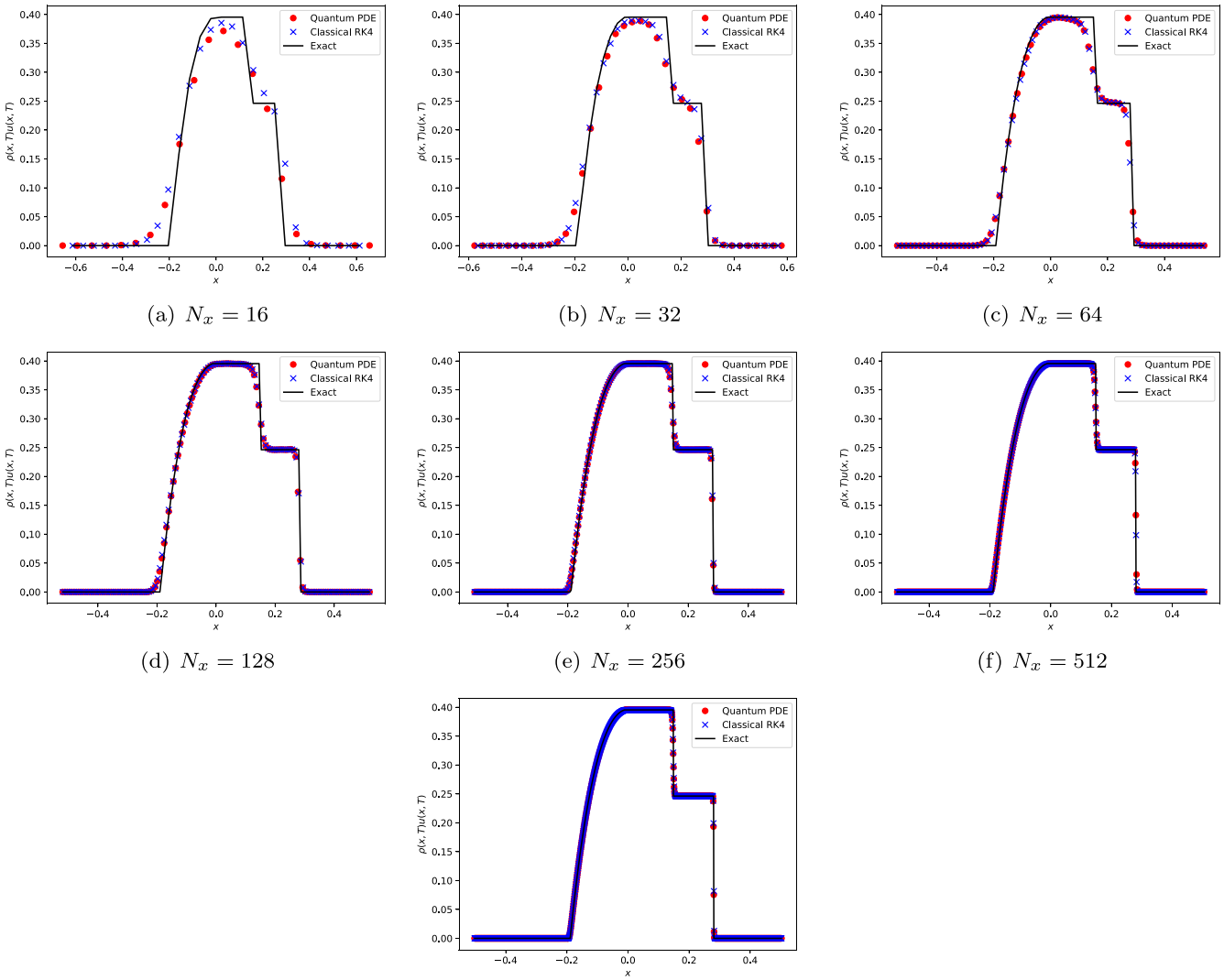
We close with some remarks that are worth noting, even though they lie outside the scope of this paper.

Remark (i). In other work, we have shown how finite-element and spectral methods can be straightforwardly incorporated into the QPDE algorithm. We will present that work elsewhere.

Remark (ii). As noted in Ref. [1], the QPDE algorithm has up to a quadratic speedup due to a square-root reduction in the number of oracle calls. Reference [16] showed how all three quantum oracles used in the QPDE algorithm can be implemented using quantum circuits. It also cost these circuits, determining their depth, width, and number of non-Clifford gates used as a function of user-specified (1) error tolerances, and (2) algorithm success probability. With these quantum oracle circuits in hand, the QPDE algorithm is now completely specified as a quantum circuit.

Having quantum circuits for the QPDE algorithm oracles allows one to discuss circuit runtimes as a function of gate execution times, and so quantum speedup in terms of algorithm runtime, rather than oracle counts. In Ref. [16], Sec. VI, remark (b), a sufficient condition [Eq. (179)] was derived for when a quantum algorithm with quantum oracles specified as quantum circuits runs in less time and with less quantum oracle calls than a classical algorithm with classical oracles specified with classical circuits.

Remark (iii). As explained in Ref. [16], the QPDE algorithm carries out one quantum state preparation and one quantum measurement when determining the approximate value of the integral associated with a secondary subinterval. The total number of secondary subintervals is n^k, where n is the number of primary subintervals and k is the



(a) $N_x = 16$ (b) $N_x = 32$ (c) $N_x = 64$

(d) $N_x = 128$ (e) $N_x = 256$ (f) $N_x = 512$

(g) $N_x = 1024$

FIG. 5. Comparing QPDE/FV-WENO simulation results for linear momentum density against the exact solution for verification problem 2: Sod's problem. (a)–(g) plot the linear momentum density $\rho(x, T)u(x, T)$ for $T = 0.16$ for spatial discretizations using $N_x = 16$–1024 FV computational cells, respectively. The quantum simulation results are plotted as red dots, while the exact solution corresponds to the black curve.

number of recursion levels used in the time partition introduced in Kacewicz's quantum ODE algorithm (see Sec. II A). Kacewicz's requires $n^{k-1} = 1/\varepsilon_1$, where ε_1 is the error upper bound on the result returned by the QAEA [21]. Solving this expression for n allows the number of secondary subintervals to be written as

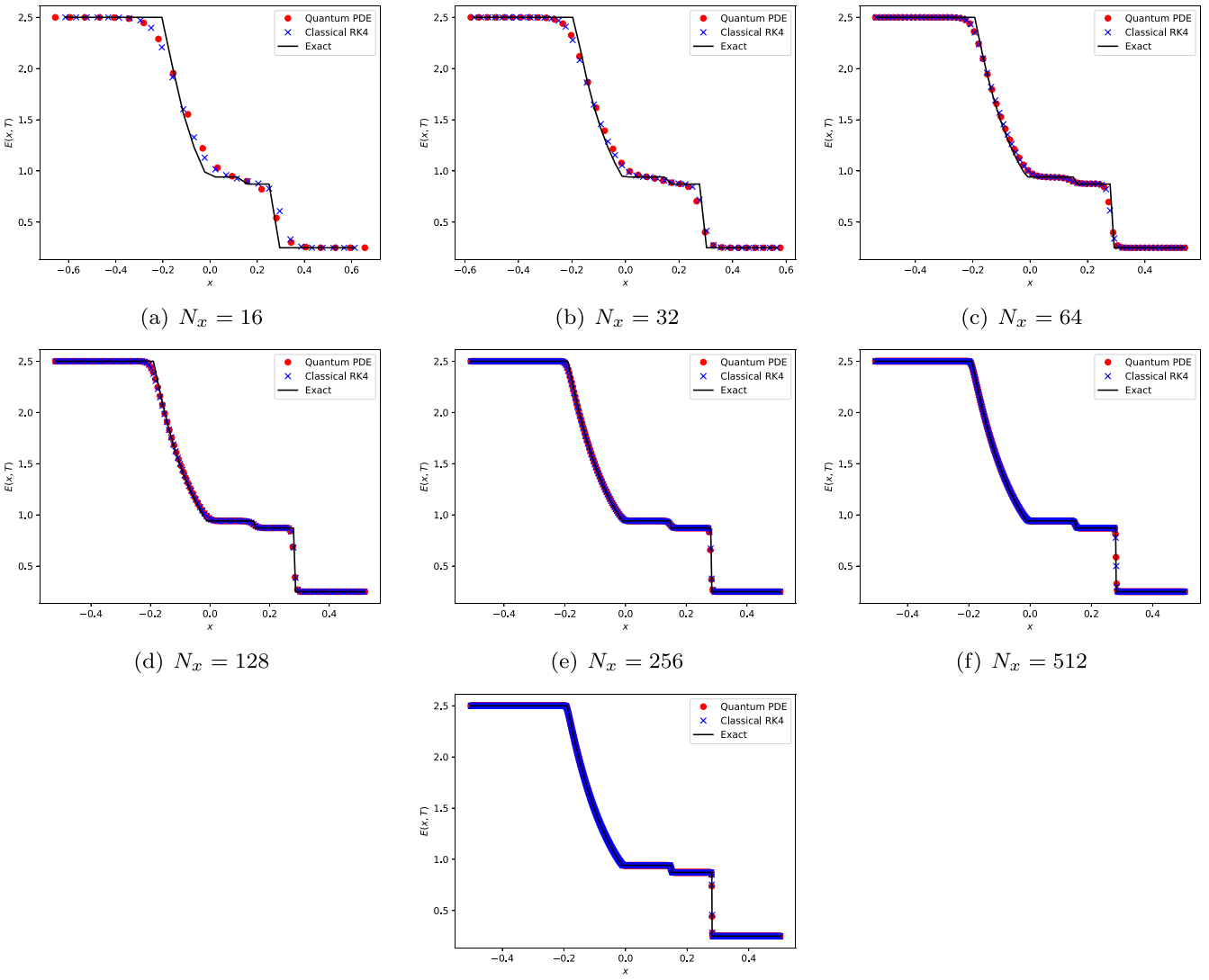
$$\begin{aligned} n^k &= (1/\varepsilon_1)(1/\varepsilon_1)^{1/(k-1)} \\ &= (1/\varepsilon_1)^{k/(k-1)}. \end{aligned} \tag{29}$$

Thus the number of state preparations and measurements used by the QPDE algorithm is $\mathcal{O}(1/\varepsilon_1)$.

Remark (iv). Note that the QPDE algorithm lends itself to massive parallel processing. This is because the algorithm gives rise to parallel data. Specifically, for each primary subinterval, the quantum circuit and quantum state used to approximate the integral associated with a particular secondary subinterval is *independent* of the circuits and states of all the other secondary subintervals. As a result, the secondary subintervals can be processed in parallel. See Ref. [25] for further discussion.

Remark (v). A major focus of this paper has been on solutions of conservation laws with discontinuities such as shock waves. A shock is an example of what is known as a weak solution of a PDE [9]. Turbulent flows provide another important example of a weak solution. We now show that turbulence can lead to flows that satisfy the conditions under which the QPDE algorithm will provide a quadratic speedup.

As shown in Refs. [1,2], the QPDE algorithm will provide a quadratic speedup for PDEs that give rise to driver functions $\mathbf{f}[\mathbf{U}(t)]$ with a small smoothness parameter $q = r + \rho \ll 1$. Here, r is the largest order time derivative of the driver function that can be taken, and ρ is the driver function's Hölder exponent. Small q thus corresponds to $r = 0$ and $\rho \ll 1$. Thus a quadratic speedup requires a continuous, nondifferentiable driver function with small Hölder exponent.



(a) $N_x = 16$ (b) $N_x = 32$ (c) $N_x = 64$

(d) $N_x = 128$ (e) $N_x = 256$ (f) $N_x = 512$

(g) $N_x = 1024$

FIG. 6. Comparing QPDE/FV-WENO simulation results for total energy density against the exact solution for verification problem 2: Sod's problem. (a)–(g) plot the total energy density $E(x, T = 0.16)$ for spatial discretizations using $N_x = 16$–1024 FV computational cells, respectively. The quantum simulation results are plotted as red dots, while the exact solution corresponds to the black curve.

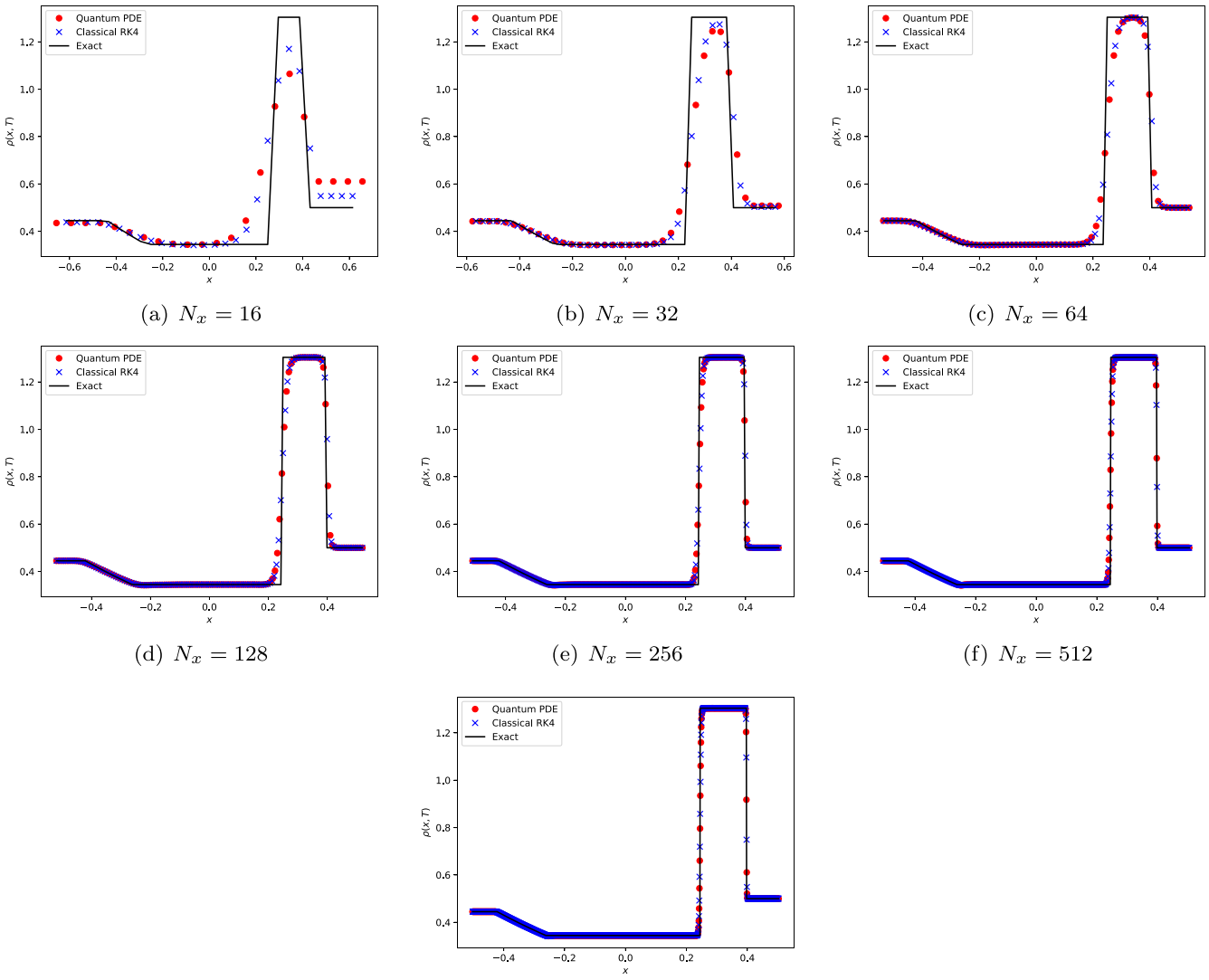
As seen in Sec. II B, the ODE driver function that results from the WENO reconstruction is based on the Lax-Friedrichs numerical flux which is well behaved for *smooth* Euler equation solutions. For the driver function to be continuous and nondifferentiable, the Euler equation solution must have these properties.

Turbulent flows are just such a family of continuous, nondifferentiable flows. Furthermore, they have been shown to have Hölder exponent $\rho \leqslant 1/3$ for length scales in the inertial range [26–29], and direct numerical simulation of turbulence [29] has yielded Hölder exponents with $0 < \rho \ll 1$. For such flows, $q \ll 1$, which is the regime where the QPDE algorithm will provide a quadratic speedup.

We noted above that a quadratic quantum speedup (in terms of oracle counts) required PDE solutions to be continuous, though nondifferentiable, with Hölder exponent $\rho \ll 1$. Thus, for a smooth solution such as the one considered in Sec. III, a quantum speedup is not expected. Whether a quantum speedup occurs for the discontinuous solutions of Secs. IV and V is unknown at this time. As explained in Sec. II, Kacewicz's quantum ODE algorithm uses a quantum computer to approximately evaluate integrals. The complexity analysis showing a quantum speedup for integration was carried out in Ref. [22] for Hölder class functions, and in Ref. [30] for Sobolev class functions. In both of these works the integrands were required to be continuous. However, the Riemann problem solutions in Sections IV and V are discontinuous, and so the analyses of Refs. [22,30] do not apply. It is thus an open question whether discontinuous PDE solutions have a quantum speedup. Further investigation is warranted.

Remark (vi). An important direction for future research on the QPDE algorithm is to look for ways to take it beyond its quadratic speedup. Such an effort is currently underway.

Remark (vii). The hybrid classical-quantum character of the QPDE algorithm provides a number of advantages that we underscore here.



(a) $N_x = 16$ (b) $N_x = 32$ (c) $N_x = 64$

(d) $N_x = 128$ (e) $N_x = 256$ (f) $N_x = 512$

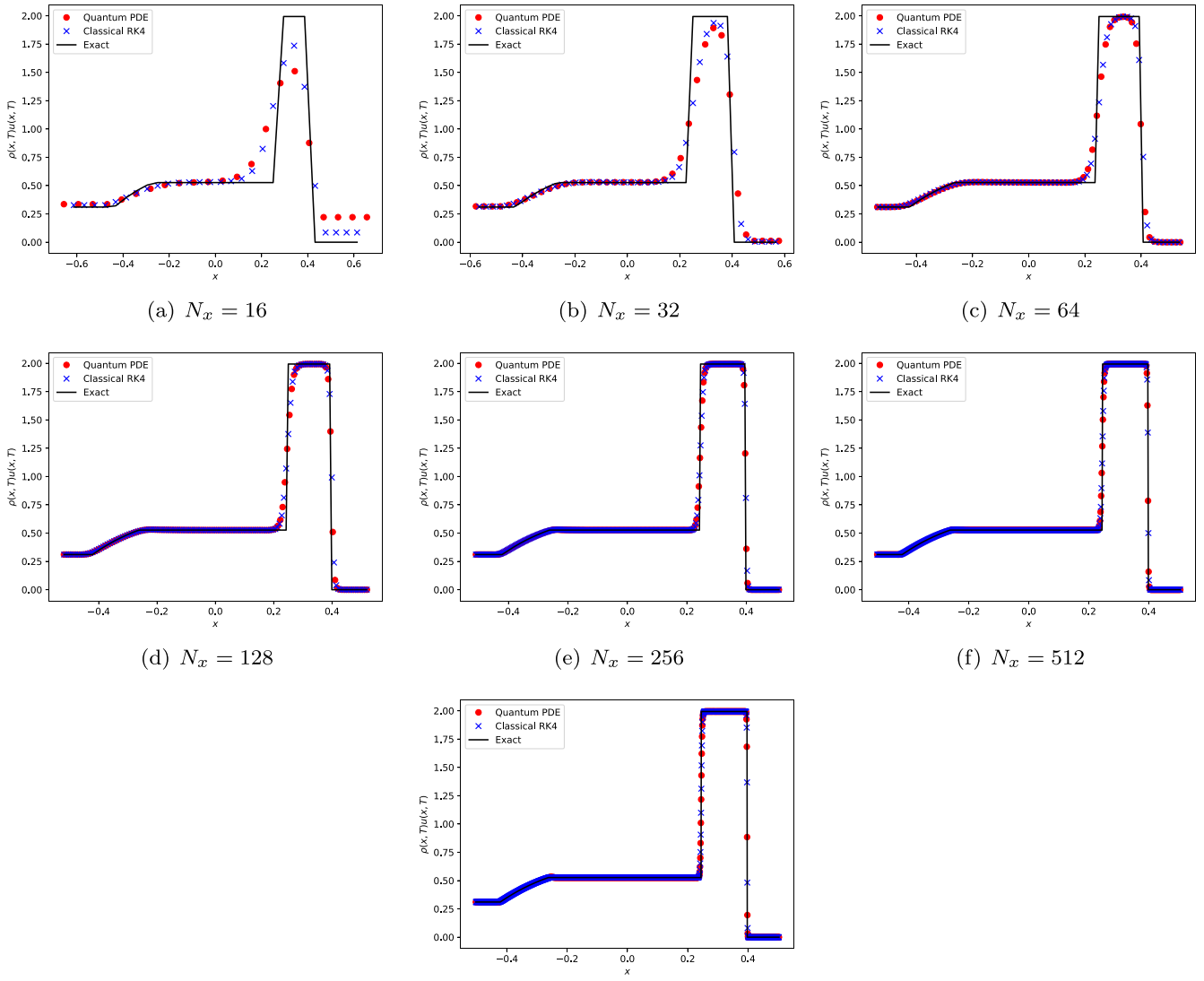
(g) $N_x = 1024$

FIG. 7. Comparing QPDE/FV-WENO simulation results for mass density against the exact solution for verification problem 3: Lax's problem. (a)–(g) plot the mass density $\rho(x, T = 0.16)$ for spatial discretizations using $N_x = 16$–1024 FV computational cells, respectively. The quantum simulation results are plotted as red dots, while the exact solution corresponds to the black curve.

As noted in Refs. [1,2], the QPDE algorithm is able to handle strongly nonlinear PDEs because it only requires a quantum computer to approximately evaluate integrals that are needed to stitch together a global solution. The nonlinearities do not present difficulties as they only impact the algebraic form of the integrand of these integrals. As shown in Ref. [16], integrand evaluation is done with a classical computer for which evaluation of a nonlinear integrand is as straightforward as evaluation of a linear one. As can be seen from the simulations results reported in this paper, the QPDE algorithm is fully capable of handling the strong nonlinearities that produce the shocks, rarefactions, and contact discontinuities appearing in Secs. III and IV.

Another important asset of the classical-quantum nature of the QPDE algorithm is that standard numerical methods for PDEs can be incorporated into it in the same manner as would be done in a classical simulation. We have seen this explicitly for FV and WENO methods in this paper. As seen in Sec. II, FV methods were incorporated just as in a classical simulation and led to the reduction of the set of PDEs to a set of ODEs. WENO methods were then incorporated as in a classical simulation to provide a numerical flux that leads to high accuracy and shock stabilization. As remarked earlier, finite-element and spectral methods were also straightforwardly incorporated into the QPDE algorithm. We see that the QPDE algorithm allows experts in, say, computational fluid dynamics, to remain experts in their fields by allowing them, when using the algorithm, to incorporate the full range of their numerical methods. They do not face the difficulty of determining how to incorporate their methods into a fully quantum algorithm whose formulation may be less amenable to this than is the case with the QPDE algorithm.

Remark (viii). Reference [16] worked out the flow of classical information between a quantum computer (QC) and a



(a) $N_x = 16$ (b) $N_x = 32$ (c) $N_x = 64$

(d) $N_x = 128$ (e) $N_x = 256$ (f) $N_x = 512$

(g) $N_x = 1024$

FIG. 8. Comparing QPDE/FV-WENO simulation results for linear momentum density against the exact solution for verification problem 3: Lax's problem. (a)–(g) plot the linear momentum density $\rho(x, T)u(x, T)$ for $T = 0.16$ for spatial discretizations using $N_x = 16$–1024 FV computational cells, respectively. The quantum simulation results are plotted as red dots, while the exact solution corresponds to the black curve.

classical computer (CC) during the execution of the QPDE algorithm. We summarize that discussion here.

(a) *Information flow from QC to CC.* As seen in Sec. II, Kacewicz's quantum ODE algorithm uses the QAEA to obtained the approximate value of integrals needed to stitch together the global approximate ODE solution. Each value is determined from the m_1 bits obtained in the QAEA's final measurement. This number of bits is related to the user-specified error tolerance ε_1 for the approximate value of the integral, and Ref. [16] showed that $m_1 \sim \log_2(1/\varepsilon_1)$. As seen in Sec. II, to each ODE there is one integral associated with each secondary subinterval. Remark (iii) above showed that there are a total of $n^k \sim 1/\varepsilon_1$ secondary subintervals, so the total number of integrals to be approximated is $\mathcal{O}(1/\varepsilon_1)$. For the Euler equations considered in this paper, there are three, hence $\mathcal{O}(1)$, ODEs per computational cell and so the total number of bits N_{Q2C} sent to the CC per computational cell is

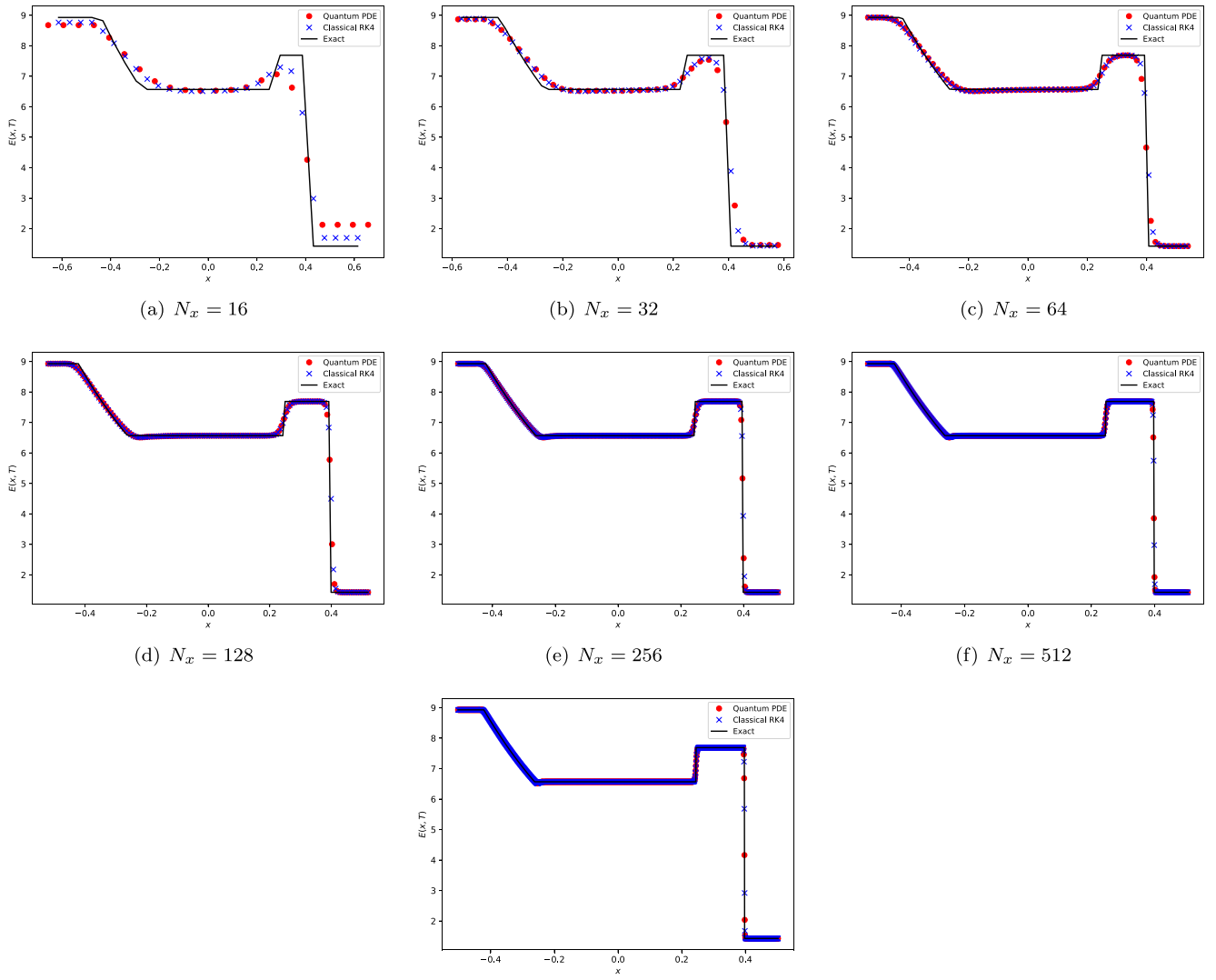
$$N_{Q2C} \sim \frac{1}{\varepsilon_1} \log_2(1/\varepsilon_1). \tag{30}$$

If we require the integral approximations to be accurate to the second decimal place, then $\varepsilon_1 \sim 10^{-2}$ and so

$$N_{Q2C} \sim 660 \text{ bits/cell} \sim 0.1 \text{ kbytes/cell}.$$

The number of computational cells needed is problem dependent. Below we consider the direct numerical simulation (DNS) of turbulence which is a notoriously difficult classical simulation problem for which we have seen [Remark (v)] that a quantum speedup is possible.

(b) *Information flow from CC to QC.* For each integral to be approximated, the CC must send the QC the weighted function values whose sum gives the integral's approximate value. As discussed in the Appendix, we use fourth-order



(a) $N_x = 16$ (b) $N_x = 32$ (c) $N_x = 64$

(d) $N_x = 128$ (e) $N_x = 256$ (f) $N_x = 512$

(g) $N_x = 1024$

FIG. 9. Comparing QPDE/FV-WENO simulation results for total energy density against the exact solution for verification problem 3: Lax's problem. (a)–(g) plot the total energy density $E(x, T = 0.16)$ for spatial discretizations using $N_x = 16 - 01024$ FV computational cells, respectively. The quantum simulation results are plotted as red dots, while the exact solution corresponds to the black curve.

Gaussian quadrature which requires that two, hence $\mathcal{O}(1)$, weighted function values be sent to the QC. Each value is approximated using $m_3 \sim \log_2(1/\varepsilon_\gamma)$ bits, where ε_γ is the error tolerance associated with approximating a real-valued fraction by an m_3-bit binary fraction. As noted above, there are $n^k \sim 1/\varepsilon_1$ integrals per ODE and $\mathcal{O}(1)$ ODEs per computational cells. Thus the total number of bits N_{C2Q} sent to the QC per computational cell is

$$N_{C2Q} \sim \frac{1}{\varepsilon_1} \log_2(1/\varepsilon_\gamma). \tag{31}$$

Setting $\varepsilon_1 \sim \varepsilon_\gamma \sim 10^{-2}$ gives

$$N_{C2Q} \sim 0.1\,\text{kbytes/cell}.$$

(c) *Number of computational cells.* We now determine the number of computational cells needed for the DNS of turbulence [31]. As noted earlier, this is a notoriously difficult classical simulation problem for which a quantum speedup is possible.

Consider then a three-spatial dimensional simulation in a computational volume of side length L_{box} with computational cells of side-length Δx equal to the Kolmogorov length $\eta = l/(\text{Re})^{3/4}$. Here, l (η) is the length scale of the largest (smallest) vortices in the simulation and $\text{Re} = ul/\nu$ is the Reynolds number, where ν is the kinematic viscosity and u is a characteristic flow speed. The number of computational cells N_{DNS} needed is then [31]

$$N_{\text{DNS}} \sim \left[\frac{L_{\text{box}}}{l}\right]^3 \text{Re}^{9/4}. \tag{32}$$

Assuming $\varepsilon_1 \sim \varepsilon_\gamma \sim \varepsilon$, Eqs. (30) and (31) give $N_{Q2C} = N_{C2Q}$. Multiplying either by N_{DNS} gives the total number of bits $N_{T,Q}$ passed between the QC and the CC over a run of the QPDE

algorithm:

$$N_{T,Q} \sim \left[\frac{L_{\text{box}}}{l}\right]^3 \text{Re}^{9/4}\left(\frac{1}{\varepsilon}\log_2(1/\varepsilon)\right). \tag{33}$$

Choosing $\varepsilon \sim 10^{-2}$, Re $= 1024$, and $L_{\text{box}}/l = 2$ so that there are (of order) eight largest scale vortices in the computational volume gives

$$N_{T,Q} \sim 3 \times 10^{10} \text{ bits} \sim 1 \text{ Gbytes}. \tag{34}$$

Note that classical DNS simulations with Re $\sim 10\,000$ are approaching the state of the art, and so our choice of Re $= 1024$ is a challenging problem for classical DNS.

To give context to the result for $N_{T,Q}$, we estimate the amount of classical information that a classical DNS simulation of turbulence must process *per time step* of the simulation. For a classical three-dimensional (3D) DNS simulation, associated with each comutational cell are five real numbers specifying the mass density, linear momentum density, and energy density. If each real value is represented by a 64-bit number, then there are 320 bits of classical information associated with each computational cell. Since there are N_{DNS} computational cells, the total number of bits of classical information $N_{T,C}$ needed to specify the state of the computation at *each time step* is

$$N_{T,C} = 320 N_{\text{DNS}}. \tag{35}$$

Assuming the same values as above for ε, Re, and L_{box}/l gives

$$N_{T,C} \sim 1 \text{ Gbytes}. \tag{36}$$

Thus a classical DNS simulation of turbulence must process at *each time step*, the same amount of classical information used in an *entire run* of the QPDE algorithm. This is an important advantage for the QPDE algorithm that supplements the quantum speedup noted above in Remark (v).

Remark (ix). In this paper we have focused on showing how a fifthorder WENO approximation can be incorporated into the QPDE algorithm. Just how much accuracy is necessary is clearly problem dependent. It is worth recalling that the fourth-order Runge-Kutta approximation is a classical workhorse because it provides a good balance of accuracy, stability, and computational workload for a wide variety of applications. The fifth-order WENO approximation provides a similar balance and range of applications, hence our focus on it in this paper. We now discuss how the order of accuracy impacts the quantum and classical workloads associated with implementing the QPDE/FV-WENO algorithm.

As shown in Ref. [16], the quantum resources required by the QPDE algorithm are determined by three errors and the algorithm success probability $1 - \delta$. The three errors are as follows: (1) the error ε_1 in the QAEA estimate of the finite sum that approximates the value of a definite integral; (2) the error ε_d in approximating the value of a definite integral by a finite sum; and (3) the error ε_γ in approximating the integrand of a definite integral by an m-bit approximation.

As seen in Sec. II A, there is one integral *per secondary subinterval, per ODE, per grid point*. As explained in Remark (iii) above, the total number of secondary subintervals $n^k = \mathcal{O}(1/\varepsilon_1)$ which is thus determined by the QAEA error listed above. Thus the quantum resources needed *per ODE, per grid point* are determined by the three errors listed above and the algorithm success probability. Furthermore, as explained in Remark (viii) above, for one spatial dimension, the Euler Equations give rise to $3 = \mathcal{O}(1)$ *ODEs per grid point*. Thus the quantum resources needed *per grid point* are determined by the above three errors and the algorithm success probability.

We now examine how many grid points are needed. For a 1D spatial domain of size L, the number of grid points, $N = L/\Delta x$, is determined by the spacing between grid points Δx, which, in turn, is controlled by the order of accuracy r. Specifically, if we require all three of the above errors to be less than a desir ed error tolerance ε, then for rth-order accuracy we have $\varepsilon \sim \Delta x^r$. Solving for the grid spacing gives $\Delta x \sim \varepsilon^{1/r}$. Thus Δx increases with increasing order of accuracy r, and so the number of gridpoints decreases with increasing order of accuracy. In conjunction with the above remarks, the total amount of quantum resources needed by a QPDE simulation, subject to a given error tolerance and algorithm success probability, decreases with increasing order of accuracy r.

Finally, implementing, say, a seventh-order (ninth-order) WENO approximation requires a seven-point (nine-point) stencil. It is important to recognize that the computations associated with implementing a WENO approximation are carried out on a *classical* computer. Seventh- and ninth-order WENO approximations increase the *classical* computational costs compared to a fifth-order WENO approximation, though as just seen, the amount of quantum resources needed decreases with increasing order of accuracy.

ACKNOWLEDGMENT

F.G. would like to thank T. Howell III for continued support.

DATA AVAILABILITY

The data that support the findings of this article are openly available [32].

APPENDIX: ERROR ANALYSIS AND THEORETICAL CONVERGENCE ORDER

In this Appendix we examine the sources of error in the approximate solution of Euler's equations produced by the QPDE/FV-WENO algorithm and bound their size. We then determine the theoretical convergence order for smooth solutions which is the family of solutions of interest in Sec. III.

There are three sources of error. The first arises from the WENO reconstruction procedure presented in Sec. II C. As noted there, the QPDE/FV-WENO simulation results presented in this paper use a WENO reconstruction procedure that is fifth-order accurate in the size of a FV cell. The second is a consequence of approximating a definite integral over a secondary subinterval by a discrete sum, and the third from truncating the Taylor series for the approximate solution $\boldsymbol{\alpha}_{i,j}(t)$ to a finite number of terms. We examine the WENO reconstruction error in Appendix 1; the discretization error in Appendix 2, and the truncation error in Appendix 3. Finally,

we combine these results to obtain the theoretical convergence order in Appendix 4.

1. WENO reconstruction error

As just mentioned, the QPDE/FV-WENO simulation results presented in this paper make use of the fifth-order accurate (in the size Δx of a FV cell) WENO reconstruction procedure presented in Sec. II C. The associated error ε_w thus satisfies

$$\varepsilon_w = \mathcal{O}\big(\Delta x^5\big). \tag{A1}$$

As explained in Ref. [2], the QPDE algorithm presented in Sec. II A is an explicit quantum algorithm. For such algorithms, numerical stability requires that its time partitioning be consistent with a stability condition such as the Courant-Friedrichs-Lewy (CFL) condition [33]. This condition defines a characteristic timescale $\Delta t_{\rm CFL}$, which for the Euler equations is [11]

$$\Delta t_{\rm CFL} = \min_{x,t}\left(C_0 \frac{\Delta x}{a(x,t)}\right). \tag{A2}$$

Here, (i) C_0 is the CFL constant whose value depends on the specific verification problem considered (see Secs. III–V) and is $\mathcal{O}(C_0) = 1$ in all cases, and (ii) $a(x,t)$ is the local speed of sound. The time $\Delta t_{\rm CFL}$ is determined by the maximum value of the local speed of sound $a_{\rm max}$. The local speed of sound is $a(x,t) = u(x,t) \pm c$, and $c = \sqrt{\gamma p/\rho}$. For the problem considered in Sec. III where $\gamma = 1.4$, $p = 1$, $u = 1$, and $\rho_{\rm min} = 0.8$, we have $a_{\rm max} = 2.323$ and so

$$\Delta t_{\rm CFL} = \mathcal{O}(\Delta x). \tag{A3}$$

Recall that the shortest timescale in the QPDE time partition is the duration of a secondary subinterval $\bar{h}$. Numerical stability thus requires $\bar{h} \leqslant \Delta t_{\rm CFL}$, and so

$$\bar{h} = \mathcal{O}(\Delta x).$$

The WENO error is then

$$\varepsilon_w = \mathcal{O}\big(\bar{h}^5\big). \tag{A4}$$

2. Discretization error

As discussed in Sec. II A, the QPDE/FV-WENO algorithm uses a quantum computer to approximately evaluate the definite integral appearing in Eq. (5). This is done by breaking up that integral over the ith primary subinterval T_i into a sum of integrals over the secondary subintervals $T_{i,j}$:

$$\int_{t_i}^{t_{i+1}} dt\, \mathbf{f}[\boldsymbol{\alpha}_i(t)] = \sum_{j=0}^{N_k-1} \int_{t_{i,j}}^{t_{i,j+1}} dt\, \mathbf{f}[\boldsymbol{\alpha}_{i,j}(t)]. \tag{A5}$$

A quantum computer is used to approximately evaluate the integrals over the $T_{i,j}$. To simplify the following discussion and unclutter the notation, we suppress the vector character of the driver function $\mathbf{f}$ and the approximate solution $\boldsymbol{\alpha}_{i,j}$, and the subscripts i, j on the approximate solution labeling the secondary subintervals. We thus focus on definite integrals of the form

$$\mathcal{I}_f = \int_{t_0}^{t_f} dt\, f[\alpha(t)]. \tag{A6}$$

As explained in Ref. [16], the rules of quantum mechanics require the integrand f to take values in the range [0,1]. This is not a serious restriction as f is, by construction, continuous over the secondary subinterval $T_{i,j}$ and so has a finite maximum (minimum) value $f_{\rm max}$ ($f_{\rm min}$) on $T_{i,j}$. Defining the function $g[\alpha(t)]$ as

$$g[\alpha(t)] \equiv \frac{f[\alpha(t)] - f_{\rm min}}{f_{\rm max} - f_{\rm min}}, \tag{A7}$$

we see that it takes values in the desired range [0,1]. We then use a quantum computer to approximately evaluate the integral

$$\mathcal{I}_g = \int_{t_0}^{t_f} dt\, g[\alpha(t)]. \tag{A8}$$

It follows from the above definitions that the desired integral $\mathcal{I}_f$ is given by

$$\mathcal{I}_f = f_{\rm min}\, \bar{h} + \mathcal{I}_g\, \Delta f, \tag{A9}$$

where $\Delta f \equiv f_{\rm max} - f_{\rm min}$, and recall that $\bar{h} = t_{i,j+1} - t_{i,j}$. It proves convenient to map $T_{i,j}$ into the interval $[-1, 1]$ via the linear transformation

$$\tau = \frac{2}{\bar{h}}(t - t_{i,j}) - 1 \tag{A10}$$

so that

$$\mathcal{I}_g = \frac{\bar{h}}{2} \int_{-1}^{1} d\tau\, g[\bar{\alpha}(\tau)], \tag{A11}$$

where $\bar{\alpha}(\tau) \equiv \alpha[t(\tau)]$. As discussed in Ref. [16], Kacewicz [18] approximates the integral in Eq. (A11) using a Riemann sum containing N_{k-1} values of the integrand g, and requires $N_{k-1} = 1/\varepsilon_1$. Here, ε_1 is the upper bound on the error of the value returned by the QAEA for the Riemann sum [21]. Its value is set by the user and provides the desired error tolerance on the approximate value of the Riemann sum. The error in $\mathcal{I}_g$ is then $\mathcal{O}(\bar{h}/N_{k-1}) = \varepsilon_1 \mathcal{O}(\bar{h}) = \mathcal{O}(\bar{h})$ which is first-order accurate in $\bar{h}$. To obtain arbitrarily higher-order accuracy we replace Kacewicz' Riemann sum approximation with a Gaussian quadrature [19] approximation,

$$\mathcal{I}_g \to I_g = \frac{\bar{h}}{2} \sum_{p=1}^{\bar{n}} \omega_p\, g[\bar{\alpha}(\tau_p)], \tag{A12}$$

where $\{\omega_p\}$ are the weights and $\{\tau_p\}$ are the roots of the $\bar{n}$th-order Legendre polynomial. The error in the $\bar{n}$th-order Gaussian quadrature approximation of the integral (over a secondary subinterval) appearing in Eq. (A11) is $\mathcal{O}(\bar{h}^{2\bar{n}})$ [19], and so the error in $\mathcal{I}_g$ is $\varepsilon_{i,j} = \mathcal{O}(\bar{h}^{2\bar{n}+1})$.

From Eq. (A5) and the continuity of $\boldsymbol{\alpha}_i(t)$ over the primary subinterval T_i, the error ε_i in the integral over T_i is the sum of the errors $\varepsilon_{i,j}$ in the integrals over all secondary subintervals $T_{i,j}$:

$$\varepsilon_i = \sum_{j=0}^{N_k-1} \varepsilon_{i,j} = N_k \mathcal{O}(\bar{h}^{2\bar{n}+1}). \tag{A13}$$

Recalling that $h = N_k \bar{h}$ and $T = nh$, we can rewrite Eq. (A13) as

$$\varepsilon_i = \frac{T}{n}\mathcal{O}(\bar{h}^{2\bar{n}}). \tag{A14}$$

The discretization error in the approximate solution at the final time $t = T$ now follows in two steps. First, by iterating Eq. (5), we obtain

$$\mathbf{y}_{n-1} = \mathbf{y}_0 + \sum_{i=0}^{n-2} \int_{t_i}^{t_{i+1}} dt\, \mathbf{f}[\boldsymbol{\alpha}_i(t)]. \tag{A15}$$

Recall from Sec. II A that $\mathbf{y}_0$ is equal to the initial condition for the system of ODEs and so is error free. The error $|\delta \mathbf{y}_{n-1}|$ is then due solely to the error in the integrals appearing in Eq. (A15):

$$|\delta \mathbf{y}_{n-1}| = \sum_{i=0}^{n-2} \varepsilon_i = (n-1)\left[\frac{T}{n}\mathcal{O}\left(\bar{h}^{2\bar{n}}\right)\right]. \tag{A16}$$

As noted in Sec. II A, and explained in more detail in Ref. [16], knowing $\mathbf{y}_{n-1}$ determines the approximate solution $\boldsymbol{\alpha}_{n-1}(t)$. In the second step, we follow the procedure Kacewicz used to derive Eq. (5) (see also Appendix 3) to derive an equation for $\boldsymbol{\alpha}_{n-1}(T)$:

$$\boldsymbol{\alpha}_{n-1}(T) = \mathbf{y}_{n-1} + \int_{t_{n-1}}^{T} dt\, \mathbf{f}[\boldsymbol{\alpha}_{n-1}(t)]. \tag{A17}$$

The discretization error ε_d in $\boldsymbol{\alpha}_{n-1}(T)$ is then the sum of the error in $\mathbf{y}_{n-1}$ and the error in the approximate value of the integral over primary subinterval T_{n-1}:

$$\begin{aligned}\varepsilon_d &= |\delta \mathbf{y}_{n-1}| + \varepsilon_{n-1} \\ &= \frac{T}{n}[(n-1)+1]\mathcal{O}(\bar{h}^{2\bar{n}}) \\ &= \mathcal{O}(\bar{h}^{2\bar{n}}).\end{aligned} \tag{A18}$$

We see that by using Gaussian quadrature to approximate the definite integrals of interest, the discretization error can be made as small as desired by an appropriate choice of $\bar{n}$. For example, choosing $\bar{n} = 2$ produces a fourth-order accurate approximation in $\bar{h}$.

3. Truncation error

As mentioned in Sec. II A, and discussed in detail in Ref. [16], the system of nonlinear PDEs of interest is reduced to a system of nonlinear ODEs by the discretization of space. Taylor's method is used to construct approximate solutions $\{\boldsymbol{\alpha}_{i,j}(I,t)\}$ over the secondary subintervals $\{T_{i,j}\}$, where I labels the FV cells. We will suppress the label I in the remainder of this Appendix. The approximate solution $\boldsymbol{\alpha}_{i,j}(t)$ is a truncation of the Taylor series representation of the exact solution $\mathbf{U}(t)$ about time $t_{i,j}$, keeping only the first $r+1$ terms:

$$\begin{aligned}\boldsymbol{\alpha}_{i,j}(t) = \boldsymbol{\alpha}_{i,j}(t_{i,j}) + \sum_{\nu=0}^{r} \frac{1}{(\nu+1)!}\frac{d^\nu \mathbf{f}}{dt^\nu}\bigg|_{\boldsymbol{\alpha}_{i,j}(t_{i,j})} (t - t_{i,j})^{\nu+1} \\ + \mathcal{O}(\bar{h}^{r+2}).\end{aligned} \tag{A19}$$

Our task is to determine the truncation error ε_t in the global approximate solution $\boldsymbol{\alpha}(t = T)$.

We begin with our system of ODEs [Eq. (4)]. As in Appendix 2, to simplify the discussion and to unclutter the notation, we suppress the vector character of the driver function $\mathbf{f}$ and the approximate solution $\boldsymbol{\alpha}_{i,j}$, as well as the secondary subinterval labels i and j on the approximate solution. We thus focus on the ODE

$$\frac{dU}{dt} = f[U], \tag{A20}$$

where $U(t)$ is the exact solution. Integrating over the ith primary subinterval T_i. and breaking up this integration into one over secondary subintervals $T_{i,j}$ gives

$$U(t_{i+1}) = U(t_i) + \sum_{j=0}^{N_k-1} \int_{t_i}^{t_{i+1}} dt\, f[U]. \tag{A21}$$

The Taylor series for $f[U]$, expanded about the approximate solution α (t), is

$$f[U] = f[\alpha] + \frac{df}{dt}\bigg|_{\alpha} (U - \alpha) + \mathcal{O}(|U-\alpha|^2). \tag{A22}$$

Inserting this into Eq. (A21) gives

$$\begin{aligned}U(t_{i+1}) = U(t_i) + \sum_{j=0}^{N_k-1} \int_{t_{i,j}}^{t_{i,j+1}} dt\, f[\alpha] \\ + \sum_{j=0}^{N_k-1} \int_{t_{i,j}}^{t_{i,j+1}} dt\, \frac{df}{dt}\bigg|_{\alpha(t)} [U(t) - \alpha(t)] + \cdots.\end{aligned} \tag{A23}$$

Note that Kacewicz derives Eq. (5) by discarding the third term on the right-hand side as small, and uses that $U(t_i) \approx y_i$ and $U(t_{i+1}) \approx y_{i+1}$ to replace $U(t_{i+1})$ $[U(t_i)]$ by y_{i+1} (y_i). For purposes of this Appendix, we want to estimate the magnitude of the discarded term as it provides a measure of the error due to using the truncated Taylor series $\alpha(t)$ in lieu of the exact solution $U(t)$.

We begin by writing

$$|U(t) - \alpha(t)| = |\alpha_*(t) - \alpha(t)|,$$

where $\alpha_*(t)$ is the untruncated Taylor series which is equal to $U(t)$ for smooth solutions. Note that this is the relevant setting for the solution of the Euler equations considered in Sec. III. Now

$$\alpha_*(t) = \alpha(t) + R_\alpha(t),$$

where $R_\alpha(t)$ is the Taylor series remainder [19]. Thus

$$|\alpha_*(t) - \alpha(t)| = |R_\alpha(t)| \leqslant C_R \bar{h}^{r+2}, \tag{A24}$$

since $\alpha(t)$ is limited to the first $r+1$ terms of $\alpha_*(t)$ [see Eq. (A19)]. Here, C_R is a constant.

We can now estimate the size of the discarded term in Eq. (A23). The magnitude of the integral in that term is

$$I = \left| \int_{t_{i,j}}^{t_{i,j+1}} dt \left[\frac{d}{dt} f[\alpha(t)] \right] (U(t) - \alpha(t)) \right|$$
$$\leqslant \int_{t_{i,j}}^{t_{i,j+1}} dt \left| \frac{d}{dt} f[\alpha(t)] \right| |U(t) - \alpha(t)|. \quad \text{(A25)}$$

The Taylor series construction [see Eq. (A19)] assumes that the driver function $f[U]$ has continuous derivatives up to order $r \geqslant 1$. Thus its first derivative is bounded

$$\left| \frac{d}{dt} f[\alpha(t)] \right| \leqslant C. \quad \text{(A26)}$$

Plugging Eqs. (A24) and (A26) into (A25) gives

$$I \leqslant C C_R \bar{h}^{r+2} \int_{t_{i,j}}^{t_{i,j+1}} dt = D \bar{h}^{r+3}, \quad \text{(A27)}$$

where $D \equiv C C_R$ and we have used that $t_{i,j+1} - t_{i,j} = \bar{h}$. Thus

$$I = \mathcal{O}(\bar{h}^{r+3}). \quad \text{(A28)}$$

Inserting Eq. (A27) into (A23) gives

$$U(t_{i+1}) \leqslant U(t_i) + \sum_{j=0}^{N_k-1} \int_{t_{i,j}}^{t_{i,j+1}} dt\, f[\alpha] + \sum_{j=0}^{N_k-1} D \bar{h}^{r+3}$$
$$\leqslant U(t_i) + \sum_{j=0}^{N_k-1} \int_{t_{i,j}}^{t_{i,j+1}} dt\, f[\alpha] + D N_k \bar{h}^{r+3}. \quad \text{(A29)}$$

From Eq. (A29), the error ε_i over a primary subinterval due to using $\alpha_i(t)$ rather than the exact solution $U(t)$ is

$$\varepsilon_i = \mathcal{O}(N_k \bar{h}^{r+3}) = (T/n)\mathcal{O}(\bar{h}^{r+2}). \quad \text{(A30)}$$

We now repeat the calculation given at the end of Appendix 2 to determine the truncation error ε_t in $\alpha(t = T)$. It is again the sum of the error $|\delta \mathbf{y}_{n-1}|$ and the error ε_{n-1} accumulated over the primary subinterval T_{n-1}. The error $|\delta \mathbf{y}_{n-1}|$ is calculated in the same way as there, and the result is

$$|\delta \mathbf{y}_{n-1}| = (n-1)\left(\frac{T}{n}\right)\mathcal{O}(\bar{h}^{r+2}).$$

The error ε_{n-1} follows from Eq. (A30) with $i = n-1$:

$$\varepsilon_{n-1} = \frac{T}{n}\mathcal{O}(\bar{h}^{r+2}).$$

Thus the trunctation error ε_t for the global approximate solution $\alpha(t = T)$ is

$$\varepsilon_t = |\delta \mathbf{y}_{n-1}| + \varepsilon_{n-1}$$
$$= [(n-1)+1]\left(\frac{T}{n}\right)\mathcal{O}(\bar{h}^{r+2})$$
$$= \mathcal{O}(\bar{h}^{r+2}). \quad \text{(A31)}$$

4. Theoretical convergence order

As noted in the introduction to this Appendix, there are three sources of error in the approximate solution produced by the QPDE/FV-WENO algorithm: (i) the WENO reconstruction error ε_w, (ii) the discretization error ε_d, and (iii) the truncation error ε_t, where

$$\varepsilon_w = \mathcal{O}(\bar{h}^5),$$
$$\varepsilon_d = \mathcal{O}(\bar{h}^{2\bar{n}}),$$
$$\varepsilon_t = \mathcal{O}(\bar{h}^{r+2}). \quad \text{(A32)}$$

The total error ε is then

$$\varepsilon = \varepsilon_w + \varepsilon_d + \varepsilon_t$$
$$= \mathcal{O}(\bar{h}^5) + \mathcal{O}(\bar{h}^{2\bar{n}}) + \mathcal{O}(\bar{h}^{r+2}). \quad \text{(A33)}$$

We can make ε_d and ε_t of comparable size by requiring

$$2\bar{n} = r + 2,$$

or

$$r = 2\bar{n} - 2. \quad \text{(A34)}$$

Thus, for example, if we require the discretization error to be fourth order so as to match the performance of a fourth-order Runge-Kutta solver, then $\bar{n} = 2$. According to Eq. (A34), for ε_t to be fourth order requires $r = 2$. In this case, our truncated Taylor series $\alpha(t)$ will contain the first $r + 1 = 3$ terms of $\alpha_*(t)$. Note, however, that for the simulations presented in Sec. III, we set $\bar{n} = 2$ and $r = 3$. From Eq. (A32), we see that ε_w and ε_t are fifth-order accurate and so ε_d, which is fourth-order accurate, is the dominant error. Thus the total error is $\varepsilon = \mathcal{O}(\varepsilon_d) = \mathcal{O}(\bar{h}^4)$. Writing $\varepsilon(\bar{h}) = C\bar{h}^4$, the *theoretical* convergence order O^c_{th} for the results presented in Sec. III is

$$O^c_{\text{th}} = \frac{\ln\left(\frac{\varepsilon(\bar{h})}{\varepsilon(\bar{h}/2)}\right)}{\ln 2} = 4. \quad \text{(A35)}$$

[1] F. Gaitan, Finding flows of a Navier-Stokes fluid through quantum computing, npj Quantum Inf. **6**, 61 (2020).

[2] F. Gaitan, Finding solutions of the Navier-Stokes equations through quantum Computing—Recent progress, a generalization, and next steps forward, Adv. Quantum Technol. **4**, 2100055 (2021).

[3] F. Oz, R. K. S. S. Vuppala, K. Kara, and F. Gaitan, Solving Burgers' equation with quantum computing, Quantum Inf. Process. **21**, 30 (2022).

[4] B. Basu, S. Gurusamy, and F. Gaitan, A quantum algorithm for computing dispersal of submarine volcanic tephra, Phys. Fluids **36**, 036607 (2024).

[5] C. J. Myers, N. Gentile, H. Rouillard, R. Vogt, F. Graziani, and F. Gaitan, Quantum simulation of non-equilibrium Marshak waves, J. Plasma Phys. **90**, 805900601 (2024).

[6] F. Gaitan, F. Graziani, and M. Porter, Simulating nonlinear radiation diffusion through quantum computing, Int. J. Theor. Phys. **63**, 260 (2024).

[7] R. J. Leveque, *Finite Volume Methods for Hyperbolic Problems* (Cambridge University Press, New York, 2002).
[8] S. K. Godunov, A difference method for numerical calculation of discontinuous solutions of the equations of hydrodynamics, Sb. Math. **47**, 271 (1959).
[9] P. D. Lax, Weak solutions of nonlinear hyperbolic equations and their numerical computation, Commun. Pure Appl. Math. **7**, 159 (1954).
[10] P. D. Lax and B. Wendroff, Systems of conservation laws, Commun. Pure Appl. Math. **13**, 217 (1960).
[11] E. F. Toro, *Riemann Solvers and Numerical Methods for Fluid Dynamics*, 3rd ed. (Springer, New York, 2009).
[12] X. D. Liu, S. Osher, and T. Chan, Weighted essentially non-oscillatory schemes, J. Comput. Phys. **115**, 200 (1994).
[13] G. Jiang and C. W. Shu, Efficient implementation of weighted ENO schemes, J. Comput. Phys. **126**, 202 (1996).
[14] C. W. Shu, High order weighted essentially non-oscillatory schemes for convection dominated problems, SIAM Rev. **51**, 82 (2009).
[15] G. A. Sod, A survey of several finite difference methods for systems of nonlinear hyperbolic conservation laws, J. Comput. Phys. **27**, 1 (1978).
[16] F. Gaitan, Circuit implementation of oracles used in a quantum algorithm for solving nonlinear partial differential equations, Phys. Rev. A **109**, 032604 (2024)
[17] L. D. Landau and E. M. Lifshitz, *Fluid Mechanics* (Pergamon, New York, 1987).
[18] B. Kacewicz, Almost optimal solution of initial-value problems by randomized and quantum algorithms, J. Complexity **22**, 676 (2006).
[19] W. Gautschi, *Numerical Analysis* (Birkhäuser, New York, 2012).
[20] A. Iserles, *A First Course in the Numerical Analysis of Differential Equations* (Cambridge University Press, New York, 2009).
[21] G. Brassard, P. Hoyer, M. Mosca, and A. Tapp, Quantum amplitude amplification and estimation, in *Quantum Computation and Information*, edited by S. J. Lomonaco, Jr. and H. E. Brandt, Contemporary Mathematics (AMS, Providence, RI, 2000), Vol. 305, pp. 53–74.
[22] E. Novak, Quantum complexity of integration, J. Complexity **17**, 2 (2001).
[23] Y. Guo, T. Xiong, and Y. Shi, A positivity-preserving high order finite volume compact-Weno scheme for compressible Euler equations, J. Comput. Phys. **274**, 505 (2014).
[24] C. Hirsch, *Numerical Computation of Internal and External Flows, Vol. II: Computational Methods for Inviscid and Viscous Flows* (Wiley, New York, 1990).
[25] B. Basu, A. Staino, and F. Gaitan, On solving for shocks and waves using a quantum algorithm, Comput. Fluids **290**, 106559 (2025).
[26] F. Nguyen, J.-P. Laval, P. Kestener, A. Cheskidov, R. Shvydkoy, and B. Dubrulle, Local estimates of Hölder exponents in turbulent vectorfields, Phys. Rev. E **99**, 053114 (2019).
[27] F. Nguyen, J.-P. Laval, and B. Dubrulle, Characterizing most irregular small-scale structures in turbulence using local Hölder exponents, Phys. Rev. E **102**, 063105 (2020).
[28] B. Dubrulle and J. D. Gibbon, A correspondence between the multifractal model of turbulence and the Navier-Stokes equations, Philos. Trans. R. Soc. A **380**, 20210092 (2022).
[29] B. Dubrulle, Multi-fractality, universality, and singularity in turbulence, Fractal Fract. **6**, 613 (2022).
[30] S. Heinrich, Quantum integration in Sobolev classes, J. Complexity **19**, 19 (2003).
[31] P. A. Davidson, *Turbulence*, 2nd ed. (Oxford University Press, New York, 2015).
[32] The simulation code used to generate the results appearing in Figs. 1–9 can be downloaded from https://github.com/DrVogtster/QPDE-FV-WENO.
[33] R. Courant, K. O. Friedrichs, and H. Lewy, Über die partiellen differenzengleichungen der mathematischen physik, Math. Ann. **100**, 32 (1928) [English translation: On the partial difference equations of mathematical physics, IBM J. Res. Dev. **11**, 215 (1967)].